THINKING CHAINS

Conscious Blockchains and the Architecture of Verifiable Machine Cognition

---

Antje Worring & Zach Kelling

Zoo Labs Foundation

January 2026

Abstract

We introduce the *Thinking Chain*: a deterministic blockchain surrounded by a verifiable cognitive layer, governed by a single discipline—*the network may think recursively, but it may only act deterministically.* Where classical chains settle transfers and contemporary AI chains settle compute, a Thinking Chain settles *thought.* The unit it settles is the **Proof-of-Thought Receipt** (PoT): a replay-protected, signed commitment that a structured cognitive question was answered by a quorum of accountable agents and that the answer was incorporated into state only after its proof was verified. We define cognition for a blockchain operationally—perception, memory, attention, deliberation, agency, self-governance, and constitutional constraint—and realize each as a specialized chain acting as a cognitive organ (C/A/D/S/R/G/M). The load-bearing safety result is the **Bridge Law**: consensus state transitions must never depend on live thought or live cross-chain queries; instead a deterministic chain emits an *intent* (Pattern A) and later verifies a *committed receipt with proof* (Pattern B). We prove that under the Bridge Law the deterministic chain's state root is a pure function of its committed inputs, independent of the cognitive layer's liveness or latency. Inference is split into two tiers: Tier 1 is deterministic int8 in-consensus inference in which every validator computes a byte-identical output (small models, e.g. a `zen-nano` port at precompile `0x03...03`); Tier 2 is large-model off-chain inference settled by *Subsampled Cognitive Consensus*—Avalanche-style repeated random committee sampling adapted so that sampled nodes answer *structured* questions over a finite output space, produce signed receipts, and are paid. Three consensus modes cover the space of cognitive questions: exact-hash quorum, preference metastability over a finite ballot, and market aggregation. Finite outputs are the rule; prose is always hash-addressed evidence, never consensus. We give the on-chain wire formats (`InferenceIntent`, `AInferenceReceipt`, keccak–Merkle proof), the sampling and commit–reveal algorithms with their economic margins (N=5, threshold=3, eligible set $E \geq N + \max(2, N/2)$, slash withholders not dissenters), a constitutional layer that bounds recursion and forbids self-granted authority, and a six-level human-in-the-loop hierarchy. The architecture is grounded, not hypothetical: the C→A bridge, the deterministic inference precompile, the quorum-settlement VM, the model registry, and the Tier-2 provider runtime are implemented in the Lux and Hanzo codebases, and cross-VM bit-identity of small-model inference was empirically verified across two independent EVM implementations. Beluga (blg), the Zoo L3 for AI model economics, is the first prototype Thinking Chain.

Keywords: conscious blockchain, cognitive consensus, proof of thought, verifiable inference, deterministic AI, subsampled consensus, constitutional AI

Contents

1 Manifesto: Chains That Think

A blockchain is a machine for agreeing on what happened. For fifteen years that machine has agreed on exactly one kind of fact: a value moved from here to there, and the move was valid. This is a narrow form of agreement, and its narrowness is the source of its strength. Determinism is not a limitation of blockchains; it is the property that makes them worth running. Every honest node, replaying the same inputs, computes the same state. Disagreement is a bug, and the bug is always attributable.

Artificial intelligence is the opposite kind of machine. A large language model is a machine for producing plausible continuations. Replayed on different hardware, with different kernels, in different floating-point rounding regimes, it produces *different* outputs. Its strength is its capacity to generalize; its weakness, from the standpoint of consensus, is that two honest evaluations need not agree. Where the blockchain treats disagreement as a fault, the model treats it as sampling.

These two machines are usually described as incompatible, and the usual conclusion is that AI must remain off-chain—a service the chain pays for but cannot verify, an oracle whose word is taken on faith. This paper rejects that conclusion. We argue that a blockchain can host genuine machine cognition without surrendering determinism, provided the two are kept in strict separation by a single architectural law. We call the result a *Thinking Chain*.

Principle 1.1 (The Thinking Chain)**.** A Thinking Chain is a deterministic blockchain surrounded by a verifiable cognitive layer. The network may think recursively, but it may only act deterministically. Thought is permitted everywhere except inside the state-transition function; the state-transition function admits thought only as a committed, verified, replay-protected artifact.

This is not a slogan. It is a partition of the system into two regions with a one-way valve between them. In the cognitive region, agents may run arbitrary models, sample, deliberate, change their minds, disagree, and escalate. In the deterministic region—the consensus state machine—nothing happens that any honest validator could not reproduce byte-for-byte. The valve between them carries not thoughts but *receipts*: signed, structured, finite commitments that some thought occurred and reached a settled conclusion. The receipt, not the thought, crosses into state.

1.1 The Thesis: Chains Pay Each Other to Think

The economic claim of this paper is as simple as the architectural one. Today's networks pay for storage (Filecoin), for bandwidth (Helium), for general compute (Akash), and for model compute as an unverified service (Render, Bittensor). A Thinking Chain pays for *settled thought*. One chain poses a structured cognitive question—"is this transaction fraudulent?", "does this proposal satisfy the treasury policy?", "which of these three remediations is safe?"—and other chains, or quora of agents, answer it. The answer is priced, sampled, committed, verified, and paid for, exactly as a transfer is priced, ordered, executed, and finalized. The unit of account is not the byte or the FLOP but the PoT: the Proof-of-Thought Receipt.

Principle 1.2 (The Unit of Thought is a Receipt)**.** The atomic, settleable object of a cognitive network is not a model output, a prompt, or a token. It is a *Proof-of-Thought Receipt*: a signed commitment that a structured question, posed at a particular point in committed history, was answered by an accountable quorum, with a recorded confidence distribution, and that the answer was admitted to state only after its proof of inclusion was verified. Prose explanations may accompany a receipt as hash-addressed evidence; they are never the object of consensus.

Why a receipt and not the answer itself? Because the answer, taken at face value, reintroduces exactly the non-determinism we are trying to exclude. Two validators asked to "run the model" would diverge. But two validators asked to *verify a receipt*—to check a quorum's signatures, to check a Merkle proof of inclusion against committed state, to check that the answer lies in the declared finite output space—will always agree, because verification is deterministic even when generation is not. This asymmetry between generation and verification is the entire technical content of the field, and we will return to it repeatedly. It is the same asymmetry that lets a deterministic chain verify a SNARK it could never have produced, applied now to cognition instead of arithmetic.

1.2 Consciousness, Carefully

We use the word *conscious* deliberately and define it operationally in Section 3, not metaphysically. We make no claim that a Thinking Chain has phenomenal experience. We claim that it exhibits the functional architecture by which cognitive systems are usually individuated: it perceives (ingests and structures external signal), remembers (maintains addressable, queryable state of past cognition), attends (allocates bounded cognitive budget to salient questions), deliberates (samples multiple agents and aggregates their structured judgments), acts (effects state changes through its own native transactions), governs itself (amends its own parameters under constitutional constraint), and—critically—operates under a constitution it cannot unilaterally rewrite. A system with all seven properties is, for engineering purposes, a cognitive agent at the network scale. Whether that constitutes consciousness in the philosopher's sense is a question we leave open and do not need to answer to build the machine.

1.3 What This Paper Is

This is the architecture and the manifesto for a class of systems, not the specification of one chain. The first instantiation—Beluga, the Zoo L3 for AI model economics [10]—is described in Section 18 as an existence proof. The mechanisms we formalize (the Bridge Law, the PoT receipt, Subsampled Cognitive Consensus, the constitutional layer) are already realized in running code in the Lux consensus stack and the Hanzo inference runtime; we cite the implementations as we introduce each mechanism so that the reader can distinguish what is built from what is designed. Nothing in this paper is vaporware: the deterministic inference precompile, the cognitive bridge, the quorum-settlement chain, and the model registry exist, compile, and are tested. What remains is to assemble them, at scale, into the full organism this paper describes.

The remainder proceeds from why (Sections 2–3), through the unit and the organs (Sections 4–6), to the load-bearing safety law and its consequences (Sections 8–11), the cognitive economy and its governance (Sections 12–16), the node architecture and first prototype (Sections 17–18), and finally the adversarial analysis, minimum viable deployments, and open problems (Sections 19–22).

2 From Ledgers to Cognitive Networks

It is worth being precise about what each generation of distributed systems learned to agree on, because a Thinking Chain is the next step in a single progression, not a departure from it.

2.1 Three Generations of Agreement

Generation 1: agreement on balances. Bitcoin [1] established that a permissionless network can agree on an ordered log of transfers without a trusted party. The agreed-upon fact is a

UTXO set. The state-transition function is a handful of validity predicates. There is no general computation.

Generation 2: agreement on computation. Ethereum [2] generalized the agreed-upon fact from balances to the result of an arbitrary deterministic program. The EVM is a replicated state machine; every node executes every transaction and must reach the same state root. The constraint that made this possible is total determinism: the EVM forbids any operation whose result could differ across nodes (no wall-clock time, no floating point, no I/O, no nondeterministic opcodes). This constraint is not incidental. It *is* the consensus.

Generation 3: agreement on cognition. The fact a Thinking Chain agrees on is the answer to a structured cognitive question, together with the evidence that the answer was produced legitimately. This is strictly harder than Generation 2, because the natural way to answer a cognitive question—run a large model—violates the determinism that Generation 2 depends on. The contribution of this paper is to show that Generation 3 can be built *on top of* Generation 2's determinism rather than by abandoning it, by settling cognition the way Generation 2 settles a SNARK verification or an oracle report: as a verified artifact, not a re-executed computation.

2.2 Why the Obvious Approaches Fail

Three approaches are commonly proposed for putting AI on a chain. Each fails in an instructive way.

(1) Run the model in the EVM.
If the model's forward pass executed as ordinary EVM bytecode, its output would be part of consensus and every node would recompute it. This works—and is exactly Tier 1 of our design (Section 9)—but *only* for models small enough and integer-only enough that the forward pass is both cheap and bit-reproducible. A 70B-parameter model in floating point cannot be run this way: it is too expensive by many orders of magnitude, and floating-point reductions are not associative, so two GPUs produce different bits. Determinism, not cost alone, is the hard wall.

(2) Trust an oracle.
Let an off-chain service run the model and post the answer; the chain takes it on faith, perhaps with a staked bond. This scales to any model but discards verifiability: a single oracle is a single point of corruption, and a bonded oracle merely prices corruption rather than preventing it. Worse, the chain's state now depends on a value no validator can reproduce or contest, which silently breaks the replay property that makes a chain auditable.

(3) Prove the inference with a SNARK.
zkML [8] produces a succinct proof that a specific model, on a specific input, produced a specific output. This is the ideal in the limit, and we expect it to subsume Tier 2 eventually. Today it is impractical at frontier-model scale: proving a single forward pass of a multi-billion-parameter transformer is many orders of magnitude more expensive than running it, and the tooling does not yet exist for the largest models. We treat zkML as a forward-compatible upgrade path for our receipt layer (Section 22), not a present-day primitive.

The design in this paper is a fourth approach that is available *today*: split the problem by model size and reproducibility (Section 9), run the small reproducible case in consensus (approach 1), and settle the large case by economic-cryptographic quorum (a hardened, structured-output adaptation of approach 2 that recovers verifiability through redundancy, sampling, and slashing). The quorum is the bridge between today's oracle and tomorrow's SNARK.

2.3 The Inversion

Generation 2 chains are passive: they compute exactly what they are told, when they are told. A cognitive network is active: it must decide *what* to think about, *how hard*, and *when to stop*. This is the inversion that motivates the rest of the paper. A ledger needs only an execution model. A cognitive network needs, in addition, an *attention* model (which questions deserve cognitive budget; Section 12), a *deliberation* model (how to aggregate disagreeing judgments; Section 10), and a *constitution* (what the network may and may not decide to do to itself; Section 16). These are not features bolted onto a ledger; they are the new organs that distinguish a cognitive network from a database. The next section defines them.

3 Defining a Conscious Blockchain

We define cognition for a blockchain operationally: by the architectural capacities the system exhibits, not by any claim about inner experience. A network is a *conscious blockchain*—equivalently, a Thinking Chain—if it realizes all seven of the following organs, each subject to the determinism partition of Section 1.

Definition 3.1 (Conscious Blockchain)**.** A conscious blockchain is a deterministic distributed state machine equipped with a verifiable cognitive layer realizing: **(1)** perception, **(2)** memory, **(3)** attention, **(4)** deliberation, **(5)** agency, **(6)** self-governance, and **(7)** constitutional constraint, such that every effect of (1)–(6) on consensus state enters only as a committed, verified, replay-protected PoT receipt.

We take the seven in turn, each with its operational test and its mapping to the organs of Section 5.

(1) Perception. The capacity to ingest external signal and render it into structured, hash-addressed form admissible to cognition. *Operational test*: the network can answer "what did I observe at height h?" with a content-addressed record whose hash is committed to state. Perception is realized by the sensory chain (S) and by Tier-1 deterministic feature extraction (e.g. int8 embedding, Section 9), which turns a raw prompt or document into a fixed-length integer vector that every validator computes identically.

(2) Memory. The capacity to retain and retrieve prior cognition by content, not merely by address. A ledger already remembers transactions; a cognitive network must additionally remember *judgments*—which questions were asked, which answers were settled, with what confidence and dissent. *Operational test*: the network can answer "have I concluded anything about x before, and how confident was I?" Memory is realized by the memory chain (M) and grounded in the Experience Ledger's content-addressable semantic store [11].

(3) Attention. The capacity to allocate a bounded cognitive budget across competing questions, refusing to spend unboundedly. This is the organ a ledger entirely lacks. *Operational test*: under a flood of questions, the network prioritizes by salience and stake and provably terminates within budget. Attention is realized by the recursion-and-budget machinery of Section 12: every cognitive task carries a depth bound and a token budget, and the scheduler is the network's attention.

(4) Deliberation. The capacity to consult multiple agents on a question and aggregate their disagreeing structured judgments into a settled conclusion with a recorded confidence distribution. *Operational test*: the network can produce, for any settled question, the ballot of how each sampled agent voted and the aggregation rule applied. Deliberation is realized by Subsampled Cognitive Consensus (Section 10) and settled on the cognition chain (A).

(5) Agency. The capacity to effect change in its own state through native transactions, conditioned on the outcome of deliberation—to *act* on what it concludes, not merely to record conclusions. *Operational test*: a settled PoT receipt can, by the rules of a contract, trigger a state transition (release escrow, update a parameter, mint, slash) without a human pressing a button. Agency is realized by the deterministic chains (C, the governance chain G) executing on verified receipts, and is the most dangerous organ—hence the constitution.

(6) Self-governance. The capacity to amend its own parameters—fees, quorum sizes, eligible model sets, budget caps—through its own deliberative and constitutional process. *Operational test*: a parameter that was v at height h_1 is v' at height $h_2 > h_1$, and the chain can exhibit the governance receipt and timelock that authorized the change. Self-governance is realized on G (Section 15).

(7) Constitutional constraint. The capacity *not* to do certain things, including not to grant itself new powers, enforced by rules that the cognitive layer cannot unilaterally rewrite. This is the organ that makes the other six safe. *Operational test*: there exists a class of state transitions (e.g. "expand the set of actions the AI may take without human approval") that no quantity of AI deliberation alone can effect—they require a human/DAO signature plus a timelock. Constitutional constraint is realized by the rules of Section 16, enforced in the state-transition function itself.

3.1 The Determinism Partition, Restated

Definition 3.1 is meaningful only because of the partition. Each organ has a cognitive part (which may be nondeterministic, off-chain, model-driven) and a settlement part (which is deterministic, on-chain, receipt-driven). Table 1 states the partition organ by organ; it is the master table of the architecture, and every later section elaborates one of its rows.

Table 1: The determinism partition: each cognitive organ split into its nondeterministic cognitive part and its deterministic settlement part. Only the right column touches consensus state.

Organ	Cognitive part (off-consensus)	Settlement part (in consensus)
Perception	model embedding, parsing	Tier-1 int8 features; S record hash
Memory	semantic retrieval, ranking	M content-addressed commitments
Attention	salience scoring	budget/depth counters; scheduler draw
Deliberation	model sampling, debate	PoT receipt + quorum verification
Agency	plan synthesis	contract executes on verified receipt
Self-governance	proposal drafting	G vote receipt + timelock
Constitution	— (no cognitive part)	hard predicates in state transition

The constitution alone has no cognitive part: it is pure deterministic predicate, by design. The AI may *propose* amendments to it (a cognitive act recorded as a governance receipt), but the amendment takes effect only through the deterministic, human-gated, timelocked path of Section 16. The network can think about its own limits; it cannot think its way past them.

4 From Thought to Receipts: The Proof-of-Thought Schema

A PoT receipt is the object that crosses the determinism valve. It must be (i) self-describing enough that any verifier can check it without re-running the thought, (ii) bound to a specific question at a specific point in committed history, so it cannot be replayed against a different question or a different time, and (iii) finite, so that checking it is a deterministic decision procedure. This section gives the abstract schema, then the concrete on-chain encoding as already realized in the Lux cognitive bridge.

4.1 Abstract Schema

Definition 4.1 (Proof-of-Thought Receipt)**.** A PoT receipt for a cognitive task τ is a tuple

$$\mathrm{PoT}(\tau) = \langle\, id,\ q,\ m,\ o^*,\ \pi,\ W,\ \kappa,\ s,\ \sigma \,\rangle$$

where id is a deterministic task identifier binding the question to its originating chain, transaction, and caller; q is the hash of the structured question (prompt, ballot, or market terms); m is the *ModelSpec* hash identifying the cognitive model(s) admitted to answer; o^* is the canonical answer drawn from a declared finite output space $\mathcal{O}$; π is the confidence/dissent distribution over $\mathcal{O}$; W is the set (Merkle root) of winning responders and their stakes; κ is the aggregation mode (Section 11); s is the settlement status; and σ is the quorum's aggregate signature over the canonical encoding of all the above.

Three properties are forced by Definition 4.1. First, $o^* \in \mathcal{O}$ with $\mathcal{O}$ finite and declared in advance: verification can enumerate or range-check $\mathcal{O}$, which is what makes it deterministic. Second, id binds the receipt to one question at one point in history, which is the replay protection. Third, π is part of the receipt, not discarded: *the network preserves dissent.* A 3-of-5 quorum that split $3{-}2$ settles the same answer as one that was unanimous, but the receipts differ in π, and downstream policy may treat a contested conclusion differently from a unanimous one (Section 16).

4.2 Canonical Encoding (As Realized)

The abstract schema is instantiated on-chain by the `AInferenceReceipt` record of the Lux `aivmbridge` precompile. Listing 1 is the actual struct; it is a fixed-width canonical encoding (every field at a pinned byte offset) so that two verifiers hash it identically—there is exactly one byte string per logical receipt.

```
type AInferenceReceipt struct {
    Version              uint16      // wire-format version
    IntentID             [32]byte    // binds receipt to one C->A intent
    TaskID               [32]byte    // A-Chain quorum task id
    CChainID             [32]byte    // originating deterministic chain
    AChainID             [32]byte    // settling cognition chain
    Requester            Address     // account that posed the question
    ModelSpecHash        [32]byte    // admitted cognitive model identity
    PromptHash           [32]byte    // hash of the structured question
    CanonicalOutputHash  [32]byte    // o*: hash of the finite answer
    Status               ReceiptStatus // Completed / Failed / ...
    N                    uint16      // sampled committee size
    Threshold            uint16      // agreement threshold for settlement
    WinnersRoot          [32]byte    // Merkle root of paid responders
```

```
    OperatorsRoot        [32]byte    // Merkle root of all sampled
        operators
    FeePaid              [32]byte    // settled fee (uint256, big-endian)
    SettledAtHeight      uint64      // A-Chain height of settlement
}
```

Listing 1: Canonical PoT receipt: the `AInferenceReceipt` settled by the cognition chain and verified by the deterministic chain (Lux `aivmbridge`).

The mapping from Definition 4.1 is direct: $id = (\texttt{IntentID}, \texttt{TaskID}, \texttt{CChainID}, \texttt{AChainID})$, $q = \texttt{PromptHash}$, $m = \texttt{ModelSpecHash}$, $o^* = \texttt{CanonicalOutputHash}$, $(N, \texttt{Threshold})$ and the two roots witness W and the sampled set, $s = \texttt{Status}$, and the confidence distribution π is recoverable from the revealed ballots committed under `OperatorsRoot`. The aggregate signature σ is supplied alongside the receipt at verification time. The canonical output is stored as a *hash*, not the bytes: the answer itself, and any accompanying prose, is hash-addressed evidence retrievable off-chain (Section 11); only its commitment lives in the receipt.

4.3 Receipts Compose

Because a receipt is itself a finite, hash-addressed object, it is admissible as evidence to a *further* cognitive task. A deliberation can take prior PoT receipts as inputs ("given that we previously concluded A with confidence 0.9 and B with confidence 0.4, is C safe?"). This is what makes recursive thinking possible (Section 12) and what makes memory cognitively useful rather than a mere log (Section 5). It is also why bounded recursion is a constitutional requirement: composing receipts without a depth bound is how a network thinks itself into an unbounded spend.

Principle 4.1 (Receipts settle; thoughts do not)**.** The settleable artifact is always the receipt, never the underlying generation. Consequently every cognitive capability of the network is expressed as: pose a structured question, sample accountable answerers, commit and reveal their finite responses, aggregate to a canonical answer with preserved dissent, sign, and verify the resulting receipt against committed state before any deterministic effect. Everything in the remainder of this paper is an elaboration of one stage of this pipeline.

5 Specialized Chains as Cognitive Organs

A monolithic chain that tried to do everything—execute contracts, run inference, store memory, sample committees, govern itself—would couple concerns that have opposite requirements. Execution wants determinism and low latency; inference wants throughput and tolerates nondeterminism; memory wants cheap large storage; governance wants slow, deliberate, human-paced finality. We therefore decompose the organism into specialized chains, each an organ optimized for one cognitive function, communicating only by committed receipts and proofs. The decomposition is orthogonal: each organ is independently complete and replaceable, and no organ reaches into another's state except through the Bridge Law (Section 8).

5.1 The Seven Organs

C — Cortex (deterministic execution).
The state machine. A standard deterministic EVM chain where contracts execute, value moves, and receipts are verified. C is where agency lives: a verified PoT receipt triggers a state transition

here. C *never* runs large-model inference or queries another chain's live state; it only emits intents and verifies committed receipts. It is the only organ whose state is the canonical ledger.

A — Cognition (deliberation & settlement).
The deliberation organ. A receives structured questions (as intents imported from C), runs Subsampled Cognitive Consensus (Section 10) over a sampled committee of providers, runs commit–reveal to settle a canonical answer, and emits a PoT receipt. A is realized by the `aivm` quorum-settlement chain. Crucially, A is itself deterministic *about settlement*: given the committed commits and reveals, the winner and the receipt are a deterministic function. The nondeterminism lives in the providers' off-chain generation, which A never re-executes—it only aggregates committed finite responses.

D — Data & provenance.
The data-commons organ. Holds content-addressed datasets, their provenance, and the royalty graph linking data to the models trained on it. D answers "where did this knowledge come from?" and routes contribution rewards. It is grounded in the data-contribution mechanisms of the Zoo stack [11].

S — Sensory (perception).
The perception organ. Ingests external feeds—price oracles, document streams, sensor data, cross-chain events—normalizes them into hash-addressed observations, and (via Tier-1, Section 9) extracts deterministic features. S answers "what did the network observe, and when?"

R — Reputation (eligibility & trust).
The trust organ. Maintains, for every provider and agent, a reputation and a diversity profile that determine sampling weight and eligibility (Section 14). R is what makes Subsampled Cognitive Consensus economically sound: it ensures the sampled committee is both competent and diverse.

G — Governance (self-governance).
The self-governance organ. Hosts proposals, votes, timelocks, and the constitutional amendment process (Sections 15–16). G is deliberately slow. AI may draft and analyze proposals here, but only the deterministic, human-gated path can enact constitutional change.

M — Memory.
The long-term memory organ. Stores settled PoT receipts indexed by content, so that prior conclusions are retrievable as evidence for future deliberations (Principle 4.1). M is what distinguishes a cognitive network from a stateless oracle: it remembers what it concluded and how confidently, and it can be asked.

5.2 The Organ Network

Figure 1 shows the organism. Note that all inter-organ edges are receipts or proofs—never live queries. C is the hub of *action*; A is the hub of *thought*; the other organs feed and constrain them. The transport that carries intents and nudges between organs is ZAP (the Zoo/Lux cross-chain transport); ZAP moves messages but never moves trust—trust is established only by verifying a committed receipt at the destination. We state this as the network's transport invariant.

Principle 5.1 (Transport carries messages, not trust)**.** ZAP transports; proofs commit; receipts settle; VMs execute. A message arriving over ZAP grants no authority by its arrival; it is acted upon only after the receiving organ verifies the committed proof or receipt it references against committed

state. The transport layer is therefore untrusted and may be lossy, reordered, or adversarial without affecting safety—only liveness.

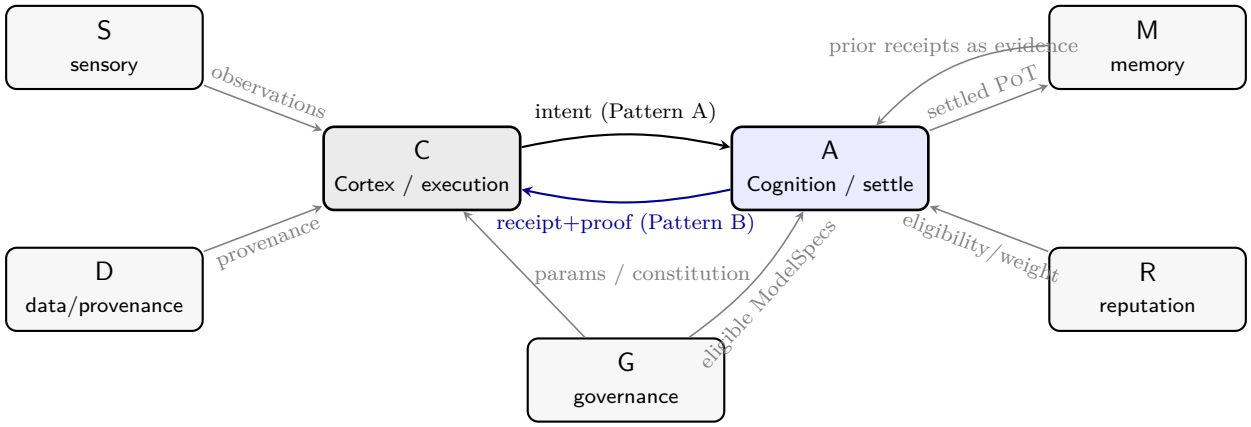

Figure 1: The cognitive organism. C (action) and A (thought) form the core loop, coupled *only* by intents (Pattern A) and verified receipts (Pattern B). Sensory (S) and data (D) organs feed perception; memory (M) returns prior receipts as evidence; reputation (R) sets sampling eligibility; governance (G) sets parameters and the constitution. Every edge is a committed message, never a live query (Principle 5.1).

5.3 Why Decompose

The decomposition is not architectural decoration; it is what makes each requirement satisfiable. A can tolerate high-throughput, GPU-bound, nondeterministic provider work precisely because it is *not* the canonical ledger—its only consensus duty is to aggregate committed finite responses, which is cheap and deterministic. C can remain a strict, fast, auditable EVM precisely because it offloads all thinking to A and only ever verifies the result. G can be slow and human-paced without throttling inference, because inference does not flow through it. This separation of concerns is the same principle that lets an operating system separate the scheduler from the filesystem from the network stack: one mechanism, one place, composable at the boundaries. The boundary discipline is the subject of Section 8.

6 The Cross-Chain Cognitive Economy

If chains pay each other to think, there must be a settled vocabulary for posing, fulfilling, and billing thought. The cognitive economy has exactly three message types, and they mirror a commercial transaction: a request, a fulfillment, and an invoice. All three are finite, hash-addressed, and verifiable; none carries trust by itself (Principle 5.1).

6.1 The Three Messages

`TaskRequest`
— the demand side. A requesting chain or agent poses a structured cognitive question: the question hash, the admitted `ModelSpec`, the finite output space $\mathcal{O}$, the aggregation mode κ, the committee size N and threshold, the budget (max fee and, for recursion, max depth), and a deterministic

routing tag. A `TaskRequest` is escrow-backed: the maximum fee is locked before any provider is asked to think, so no provider works for a promise.

`Receipt`
— the supply side. The PoT receipt of Section 4, emitted by A once a quorum settles the question. The receipt is the proof of fulfillment; it is what the requester verifies before acting and what the providers present to be paid.

`Invoice`
— the settlement side. The accounting record that splits the escrowed fee among the winning providers (by stake and reputation), the model owner (a royalty on the admitted `ModelSpec`), the data contributors whose data trained that model (via D's provenance graph), and the protocol. The invoice is derived deterministically from the receipt's `WinnersRoot` and D's royalty graph; it is not negotiated.

6.2 Pricing a Thought

The price of a thought is a function of its cognitive cost and the demand for it. We price a Tier-2 task by the committee size, the model scale, the question's structured complexity (output-space size and sequence length), and a demand-surge term, paralleling the inference pricing already deployed for the first prototype (Section 18):

$$fee(\tau) \;=\; N \cdot p_m \cdot \left(\frac{|q|}{q_0}\right)^{1.2} \cdot \big(1 + \beta\, u\big), \tag{1}$$

where N is the committee size (each member is paid to think), p_m is the admitted model's base price, $|q|$ is the question's token length normalized to a reference q_0, the super-linear exponent reflects attention's quadratic cost, u is current cognitive-load utilization on A, and β is the surge coefficient. The escrowed maximum is $N \cdot p_m \cdot (\dots)$ at the cap; unspent escrow is refunded if fewer than N providers are paid or the task fails to reach quorum.

6.3 The Settled Loop

Figure 2 is the canonical request→pay loop that every cognitive transaction follows. It is the operational realization of Principle 4.1, drawn end to end. Each stage is a committed on-chain action except the providers' generation (stage 3, off-chain, nondeterministic), which is exactly the stage the determinism partition isolates.

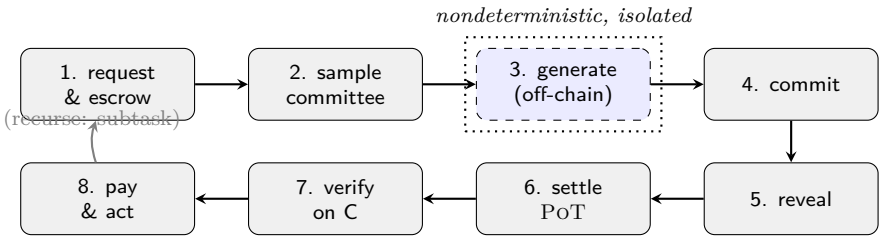

Figure 2: The cognitive transaction loop: *request* → *sample* → *commit* → *reveal* → *settle* → *verify* → *pay*. Every stage is a committed deterministic action except generation (stage 3), the single nondeterministic step, which is isolated off-chain and never enters consensus. A settled task may spawn subtasks (recursion, Section 12) subject to the inherited budget.

The loop is closed and self-funding: escrow in at stage 1, paid out at stage 8, with the protocol's cut and the model/data royalties flowing to the organs that made the thought possible. A Thinking Chain is therefore not a cost center bolted onto a ledger; it is an economy whose product is settled judgment. Sections 8 through 10 formalize the dangerous middle of this loop—the bridge from action to thought and back (stages 1, 7), and the consensus that settles the answer (stages 2–6).

7 Proof-of-AI: The AI Bitcoin

Bitcoin's durable invention is not the coin but *trustless issuance through verifiable work*: anyone may expend a scarce resource, prove the expenditure, and earn a unit of a permissionless asset, with the network's security paid for by that same expenditure. Its defect is equally plain—the work, hashing, is intrinsically useless. Proof-of-Work converts energy into security and issuance and discards the energy; the miner's output is wanted by no one but the protocol.

A Thinking Chain keeps the mechanism and removes the waste. The scarce resource expended is *settled cognition*: inference a paying agent demanded, produced under a sampled committee, and committed as a PoT receipt (Section 4). The proof of work *is* the useful work. We call this **Proof-of-AI** (PoAI)—the trustless, permissionless issuance of Nakamoto consensus, but where the act that mints the coin and secures the chain is the act of answering a real question. The receipt is the coinbase of a thought.

Principle 7.1 (Proof-of-AI)**.** In Proof-of-AI the security budget *is* the revenue. Where Proof-of-Work spends a real resource to produce only security, Proof-of-AI spends bonded compute on cognition that is sold to the demand side. The network is secured by doing the work the world is already paying for, not by work performed in addition to it.

7.1 The Correspondence

Every load-bearing part of Nakamoto consensus has a Proof-of-AI analog, and in each, the analog produces value where the original produced waste.

Table 2: Bitcoin and Proof-of-AI, mechanism for mechanism. Two rows—validation and issuance—carry the thesis.

	Bitcoin (PoW)	Thinking Chain (PoAI)
Scarce input	energy (hashing)	bonded compute on demanded inference
Unit of issuance	block reward	PoT receipt: one settled cognition
Validation	re-hash the header	re-derive the Tier-1 output, byte-identical (Section 9)
Sybil resistance	cost of hashpower	bond + eligible-set margin + reputation
Bad-actor response	— (no recovery)	fraud proof + reputation decay (§7.2)
Finality	probabilistic (longest chain)	quorum-settled, then deterministic
Issuance source	protocol inflation on a clock	demand-side escrow (Section 6)
Product of mining	none (discarded)	the answer the requester buys

Validation is the heart of it. Just as any node re-hashes a Bitcoin header to check a claimed solution, any validator on a Thinking Chain re-runs the Tier-1 deterministic model and obtains

a byte-identical output (Section 9)—the cognitive equivalent of the hash everyone can recompute, which is precisely what lets strangers agree on a thought without trusting the thinker. *Issuance* is the other half: Bitcoin mints supply against a clock; a Thinking Chain pays providers from escrow funded by the agents who demanded the cognition (Section 6). The coin is not printed; it is earned against demand.

7.2 Correctness Without Slashing

Proof-of-Work needs no slashing because a wrong answer simply fails to verify—the network ignores it at zero cost. Proof-of-AI inherits this property and so needs no slashing as its primary defense either. Correctness comes, in order, from three layers that escalate in cost only as needed:

1. **Reproduction (free).** A Tier-1 receipt is byte-reproducible: any party re-derives the output and a mismatch is self-evident. A false receipt is not punished—it is *refuted*. This is an optimistic fraud proof, identical in spirit to a rollup's: one honest re-execution invalidates a lie, and the would-be forger is paid nothing.

2. **Reputation (continuous).** For Tier-2, where outputs are not bit-reproducible, providers earn by *measured, ongoing answer quality* rather than by avoiding a penalty—the Bittensor lesson. Agreement with quorum and with later-confirmed truth raises a provider's weight and routing share (Section 14); persistent divergence decays it. A bad actor is not slashed; it is *routed around* and stops earning. Non-payment plus reputation decay is the disincentive.

3. **Bond (optional, layered).** The hard slashing table of Section 19 is retained only as an optional backstop for the handful of *deterministically detectable* faults—equivocation, double-signing—where capital at risk adds defense in depth. It is a layer a deployment may enable, not the foundation. The forgery floor (Section 19) still bounds the cost of a fabricated high-value receipt; under PoAI that floor is reputation-weighted, not slash-dependent.

The shift matters: a network that secures itself by *measuring useful work* rather than by *threatening to burn capital* is open to heterogeneous, intermittent, permissionless providers—exactly the supply that a global compute market must admit.

7.3 The Compute Market Underneath: the Hamiltonian Market Maker

Bitcoin's difficulty adjustment prices a single homogeneous resource (hashpower). Intelligence is not homogeneous: it is produced on GPUs, CPUs, unified-memory Apple silicon, and accelerators that differ by orders of magnitude in memory, bandwidth, and throughput. Proof-of-AI therefore replaces difficulty with a market. The price of a thought is set by the **Hanzo Hamiltonian Market Maker** (HMM), a physics-based automated market maker over the cotangent bundle of a heterogeneous-compute reserve manifold—GPU-hours, VRAM, CPU, bandwidth, storage. Reserves q_i and demand-credit p_i evolve under Hamilton's equations from a Hamiltonian that generalizes the constant-product invariant with a harmonic mean-reversion term, so prices are continuous, capital-efficient, and self-stabilizing; a Hidden-Markov regime detector (FAIR/LOADED) and active-inference routing place each thought on the cheapest capable hardware. "Difficulty" becomes a living market for heterogeneous cognition, which is what lets PoAI mine across the messy real world of compute rather than a single hash. The escrow price of Equation 1 is the user-facing quote; the HMM is the clearing engine beneath it.

7.4 The Compounding Engine

Bitcoin's supply is inert: a coin, once mined, produces nothing. A Thinking Chain's output is *generative*, and this is the decisive economic difference. PoT receipts compose (Section 12): a settled thought is evidence for the next, so the network deliberates recursively and accumulates a deepening tree of committed reasoning. That tree is not discarded—settled cognition and its provenance feed model improvement, so the network *learns from what it settles*. Each community runs its own Thinking Chain, and each such network is **recursively self-upgrading**: it continually produces new content, intellectual property, and research, every artifact minted as an on-chain asset with provenance and perpetual royalties (the `Invoice` split of Section 6 pays the model owner and the data contributors whose work made the thought possible). The asset's backing therefore *grows as the network thinks*—a property no proof-of-work or proof-of-stake chain possesses. Value compounds because cognition compounds.

7.5 Why This May Be the Most Valuable Protocol Ever Built

The claim is large and we state it plainly: a Thinking Chain may become the most valuable protocol in the history of blockchains. The argument is not hype but a chain of the properties above.

- **A larger, productive market.** Bitcoin monetized scarcity—a store of value. Proof-of-AI monetizes *verifiable intelligence* and its compounding output of IP and research. The total demand for settled, actionable cognition dwarfs the demand for a static reserve asset, and unlike a reserve asset it is consumed and re-demanded continuously.
- **The base money of the machine economy.** As autonomous agents become the dominant economic actors, every agent that needs a thought it can act on must pay through a settlement layer that makes thought verifiable. The protocol that settles machine cognition becomes the agents' reserve asset and unit of account by the same logic that made the dollar the unit of human trade.
- **Security that is revenue, issuance that is demand-backed.** PoAI spends nothing the market does not already buy (Principle 7.1); the coin is earned against demand, not printed against a clock. This is sounder money than Proof-of-Work, not weaker.
- **A flywheel that compounds the backing.** More providers → cheaper and better cognition (priced by the HMM) → more demand → more fees and better self-upgraded models → more providers. Each turn the network's accumulated IP and research—its backing—grows (Section 7.4).
- **Value with values.** In its first instantiation, Beluga, a constitutional conservation tithe (Section 18.1) routes a fixed slice of every settled cognition to the commons. The protocol can have a conscience without sacrificing its economics.

A protocol whose block production *is* the most-demanded work of the era, whose security is its revenue, whose issuance is demand-backed, and whose output compounds into perpetual, royalty-bearing intellectual property, has a claim that no prior chain can make. Bitcoin proved that a network can mint trustless money from useless work. A Thinking Chain proposes the sequel: trustless money from the most useful work there is.

8 The Bridge Law

Everything in this paper rests on one safety discipline. We state it as a law, give the two patterns that implement it, exhibit the on-chain mechanism that enforces it, and prove the property it guarantees.

Principle 8.1 (The Bridge Law)**.** A consensus state transition must never depend on live thought or on a live cross-chain query. A deterministic chain may interact with the cognitive layer in exactly two ways: **Pattern A**, by emitting a deterministic *intent* (a committed record of a question, with no answer); and **Pattern B**, by verifying a previously *committed receipt and its proof of inclusion* against committed state. There is no third pattern. In particular, no opcode, precompile, or contract may block on, or branch on, a value that is being computed by the cognitive layer at the moment of execution.

The Bridge Law is the formal content of "the network may think recursively, but may only act deterministically." Pattern A is how the deterministic chain *asks*; Pattern B is how it *learns the answer*, but only after the answer has become a committed historical fact that every validator can verify identically. The two patterns are separated in time by the entire cognitive process: the chain emits an intent in one transaction and verifies the resulting receipt in a strictly later one, never in the same execution.

8.1 Pattern A: Submit a Deterministic Intent

Pattern A records a question without learning its answer. Listing 2 is the actual interface and intent-id construction from the Lux `aivmbridge` precompile. The key property is that `submitInferenceIntent` returns an *id*, never an output: the call is total, deterministic, and depends on nothing but its committed arguments and committed chain context.

```
// Pattern A -- ask, do not learn the answer.
function submitInferenceIntent(
    bytes32 modelSpecHash,  // admitted cognitive model
    bytes32 promptHash,     // hash of the structured question
    uint16  n,              // requested committee size  (1..MaxFanout)
    uint16  threshold,      // agreement threshold       (1..n)
    uint256 fee,            // escrowed maximum
    bytes32 routing         // deterministic routing tag
) external returns (bytes32 intentID);
```

Listing 2: Pattern A: a deterministic chain emits an intent. The call records a question and returns a deterministic id; it cannot return an answer (Lux `aivmbridge`, selector `0x10000000`).

The returned `intentID` is not random. It is a deterministic hash over the originating chain id, the originating transaction hash, the call index within that transaction, the caller, the model identity, and the prompt—so that a re-executed or reorged transaction maps to the *same* intent (idempotence) and two genuinely distinct questions get distinct ids. Determinism of the id is what lets every validator agree that this intent was emitted, without any of them knowing what the answer will be.

8.2 Pattern B: Verify a Committed Receipt

Pattern B is how the answer re-enters the deterministic world. The chain is handed a PoT receipt and a Merkle proof that the receipt is included under a committed receipts root, and it verifies—it

does not trust, and it does not query A live. Listing 3 is the actual verification interface and the proof structure.

```
// Pattern B -- learn the answer only after it is committed and proven.
function verifyInferenceReceipt(
    bytes receiptBytes,   // canonical AInferenceReceipt encoding
    bytes proofBytes      // keccak-Merkle inclusion proof
) external returns (
    bytes32 intentID,           // must match a pending intent
    bytes32 canonicalOutputHash,// o*: the settled finite answer
    uint8   status              // Completed iff fully verified
);
```

Listing 3: Pattern B: a deterministic chain verifies a committed receipt against committed state. No live query to the cognition chain occurs; verification is a pure function of the inputs (Lux `aivmbridge`, selector `0x11000000`).

```
type AInferenceProof struct {
    ReceiptRoot [32]byte   // committed root the receipt is proven under
    Path        [][32]byte // sibling hashes, leaf -> root
    Index       uint64     // leaf position (must fit in len(Path) bits)
}
```

Listing 4: The inclusion proof verified in Pattern B: a keccak–Merkle path. `VerifyMerkle` recomputes the root and rejects index/depth aliasing—it is a deterministic decision procedure.

Verification performs only deterministic checks: recompute the keccak–Merkle root from the receipt leaf and path and compare it to a committed root; confirm `intentID` matches a pending intent emitted earlier by Pattern A; confirm the receipt's `ModelSpecHash`, `PromptHash`, `Requester`, N, threshold, and chain ids match the intent; confirm `Status` is `Completed`; and enforce replay protection by retiring the pending intent on success. Every one of these is a pure function of the call's inputs and committed state. *Pattern B never depends on the cognition chain being live*—a verifier can check a receipt long after A has moved on, or even if A is offline, because it is checking committed history, not asking a question.

8.3 The Safety Theorem

The Bridge Law buys a precise, provable property: the deterministic chain's state is unaffected by the cognitive layer's nondeterminism, latency, or liveness. We formalize the state-transition function and prove it.

Let Σ be the C state space and let $\delta : \Sigma \times \mathcal{T} \to \Sigma$ be its state-transition function, where $\mathcal{T}$ is the space of transactions. Let Λ denote the cognitive layer (all of A and its providers), and let $\lambda_t \in \Lambda_{\text{live}}$ be the live cognitive state at the moment transaction t executes (model weights actually loaded, sampling RNG, GPU rounding, network latency, which providers happen to be online). A transaction's execution may read committed C state and its own committed arguments; under the Bridge Law it may additionally invoke only Pattern A and Pattern B.

Theorem 8.1 (Determinism of C under the Bridge Law)**.** *If every interaction between C and the cognitive layer obeys the Bridge Law (Principle 8.1), then for all states $\sigma \in \Sigma$ and transactions $t \in \mathcal{T}$, the successor state $\delta(\sigma, t)$ is independent of the live cognitive state λ_t. Equivalently, the C state root after applying any committed block is a pure function of committed inputs, and any two*

honest validators replaying the same committed history compute the same state root, regardless of the behavior, timing, or availability of the cognitive layer.

Proof. Consider the only points at which t's execution could come to depend on λ_t. By the Bridge Law, t touches the cognitive layer only via Pattern A or Pattern B; we show neither introduces a dependence on λ_t.

Pattern A. The call `submitInferenceIntent` computes `intentID` as a deterministic hash of (originating chain id, originating tx hash, call index, caller, `modelSpecHash`, `promptHash`) and records a pending intent; it returns this id. Every argument to the hash is either a committed argument of t or committed C context (the tx hash and call index are fixed once t is included). None is a function of λ_t: in particular no answer is produced, so no model is run during t. Hence the call's effect on σ and its return value are functions of committed inputs alone, independent of λ_t.

Pattern B. The call `verifyInferenceReceipt` takes `receiptBytes` and `proofBytes` as ordinary committed call data of t. Its checks are: (i) recompute the keccak–Merkle root from the receipt leaf and the supplied path and compare to a root committed in C state; (ii) match the receipt's fields to a pending intent recorded in C state; (iii) check `Status`; (iv) retire the intent. Each check reads only committed call data and committed C state and applies a fixed (keccak, comparison, lookup) function. The receipt and proof were produced by Λ at some earlier time, but at the moment of t's execution they are immutable bytes in t's call data and an immutable committed root; the call does not re-run inference and does not query Λ. A receipt that is forged, stale, or never produced does not make the call depend on λ_t; it merely makes verification *fail* deterministically (the root mismatches, or no matching pending intent exists), and a failed verify is a deterministic outcome of committed inputs.

No other interaction with Λ exists, by hypothesis. Therefore t's read set and write set are functions solely of committed C state and t's committed arguments, so $\delta(\sigma, t)$ is independent of λ_t. By induction over the transactions of a block and over blocks, the post-block state root is a pure function of committed history. Two honest validators replaying the same committed history apply the same δ to the same inputs and obtain the same root, irrespective of Λ's behavior, timing, or availability. □ □

Corollary 8.2 (Liveness/safety decoupling)**.** *The availability of the cognitive layer affects only* C liveness *of cognition-dependent actions (a receipt that is never produced is a receipt that is never verified, so the dependent action never fires), never* C safety*. A halted, slow, or partially adversarial cognitive layer cannot fork* C *or corrupt its state; in the worst case, cognition-gated actions simply wait.*

Theorem 8.1 is the license for everything that follows. Because the deterministic chain is provably insulated, we are free to make the cognitive layer as nondeterministic, as model-heavy, and as economically intricate as we like—sampling committees, large language models, market aggregation, recursive subtasks—without ever endangering the property that makes a blockchain a blockchain. The cognitive layer is where we relax determinism; Theorem 8.1 guarantees that the relaxation stays on its own side of the valve.

9 Two-Tier Inference

The Bridge Law tells us *how* cognition re-enters state (as a verified receipt). It does not tell us *how the answer is produced*. There are two regimes, divided by a hard physical fact: whether the

model's forward pass is bit-reproducible across heterogeneous hardware. This fact, not cost, is the dividing line.

Principle 9.1 (The reproducibility line)**.** A model's forward pass may be admitted *into* consensus (every validator recomputes it) only if it is byte-identical across all honest validators' hardware. Integer-only inference with frozen requantization is byte-identical; floating-point inference is not, because floating-point reduction is not associative and differs across GPU kernels, vendors, and driver versions. Therefore: small integer models run in consensus (Tier 1); large floating-point models run off-chain and are settled by quorum (Tier 2).

9.1 Tier 1: Deterministic In-Consensus Inference

Tier 1 is the case where the model is small enough and integer-only enough that every validator can run the forward pass and obtain the *same bits*. Here inference is not bridged at all—it is an ordinary deterministic computation, settled by the same mechanism that settles an `ADD`. There is no quorum, no commit–reveal, no receipt: the model output is consensus, exactly like any other EVM result.

This is realized today by the Lux deterministic inference precompile at address `0x0300...0003` (the AI-mining range, significant byte `0x03`, suffix `0x03`). It is a pure-Go int8 transformer compiled with `CGO=0`—no C dependencies, no platform-specific math—implementing a port of a small `zen-nano`-class model. Listing 5 sketches the integer kernels; the entire pipeline (embedding gather → RMS-norm → int8 GEMM → frozen-LUT softmax attention → SwiGLU → requantize → argmax) uses only integer arithmetic with fixed right-shift requantization and frozen lookup tables, so the result is a deterministic function of the input tokens and the committed weights.

```
// Saturating int8 cast -- the requantization primitive.
func satI8(v int32) int8 { /* clamp to [-128,127] */ }

// int8 x int8 -> int8 GEMM with int32 bias and a fixed shift.
// No floating point anywhere: the result is bit-identical across
// every honest validator's hardware, which is what admits it to
// consensus.
func gemmI8(a, bt []int8, bias []int32, c []int8,
            m, n, k uint32, shift int32) {
    for i := uint32(0); i < m; i++ {
        for j := uint32(0); j < n; j++ {
            var acc int32 = bias[j]
            for p := uint32(0); p < k; p++ {
                acc += int32(a[i*k+p]) * int32(bt[j*k+p])
            }
            c[i*n+j] = satI8(acc >> shift) // frozen requant
        }
    }
}
```

Listing 5: Tier-1 deterministic kernels: saturating int8 arithmetic and int8 GEMM with a fixed requantization shift. Pure Go, `CGO=0`; every validator computes byte-identical output (Lux `precompile/inference`, `0x0300...03`).

The integer-quantization discipline follows the standard low-precision inference recipe established by gemmlowp [7] and the int8 quantization literature [6]: weights and activations are int8,

accumulation is int32, and requantization is a fixed multiplier-and-shift. The novelty here is not the arithmetic but the *requirement*: in a normal ML system int8 is an optimization, whereas in Tier 1 it is a consensus precondition—a floating-point version of the same model would be unusable on-chain not because it is slow but because it is nondeterministic.

Empirical bit-identity. The reproducibility claim is not assumed; it was verified. The same engine, fed the same input and the same committed weights, produces an *identical output hash* across two independent EVM implementations—the Go EVM and the Rust EVM. This cross-VM bit-identity is the empirical license for Tier 1: if two independently written virtual machines agree byte-for-byte, the forward pass is genuinely deterministic and safe to admit into consensus. The companion negative result is equally important: the same exercise on a large floating-point model does *not* reproduce across hardware, which is precisely why such models are confined to Tier 2.

9.2 Tier 2: Large-Model Quorum Inference

Tier 2 is everything Tier 1 cannot be: frontier-scale models, floating-point, GPU-bound, nondeterministic. These cannot be run in consensus (Principle 9.1), so they run off-chain on a network of providers and are settled by *Subsampled Cognitive Consensus* (Section 10), producing a PoT receipt verified via Pattern B. The Tier-2 provider runtime is the Hanzo engine with its FFI boundary (`hanzo-engine` / `hanzo-engine-ffi`), which hosts the large models and emits the structured, committable responses that the quorum aggregates [9].

The two tiers are not competitors; they are a routing decision made per question. A cheap, reproducible classification ("is this string a valid address?", "which of these four risk buckets?") routes to Tier 1 and is settled for free as ordinary execution. An expensive, open-ended judgment ("summarize the risk in this 50-page document and rate it") routes to Tier 2 and is settled by paid quorum. Table 3 contrasts them. The dividing line is always Principle 9.1: reproducible ⇒ Tier 1; not reproducible ⇒ Tier 2.

Table 3: The two inference tiers.

	Tier 1 (in consensus)	Tier 2 (quorum-settled)
Model scale	small (e.g. `zen-nano`)	frontier (e.g. 70B+)
Arithmetic	int8, frozen requant	floating point
Reproducible?	yes (byte-identical)	no (cross-hardware drift)
Who computes	every validator	sampled provider committee
Settlement	ordinary EVM result	PoT receipt + Pattern B
Cost to verify	nil (recomputed)	quorum + Merkle proof
Realized by	`0x0300...03` precompile	Hanzo engine + `aivm`

9.3 Why Both

A network with only Tier 1 could be fully verified but could think only small thoughts. A network with only Tier 2 could think large thoughts but would pay a quorum for every trivial classification. The two-tier design lets the network spend its consensus budget where reproducibility is free and its economic budget where reproducibility is impossible, routing each question to the cheapest mechanism that can answer it safely. This is the same instinct as a CPU's cache hierarchy: do the cheap, exact thing inline; escalate to the expensive, redundant thing only when you must. The escalation policy—which questions deserve Tier 2's paid quorum and how deep recursion may go—is the network's attention, formalized in Section 12.

10 Subsampled Cognitive Consensus

Tier-2 thoughts are produced by nondeterministic models, yet must be settled into a single canonical answer that a deterministic chain can verify. We obtain this by adapting the consensus idea that Avalanche made practical—*repeated random subsampling*—to the cognitive setting. In Avalanche [4, 3], a node repeatedly samples a small random committee from the validator set and adopts the supermajority preference, achieving metastable agreement with sublinear communication. We keep the sampling skeleton and change what the sampled nodes are asked: instead of "which block do you prefer?", a sampled provider is asked a *structured cognitive question with a finite answer*, must *commit then reveal* a signed response, and is *paid* for an honest, agreeing answer and *slashed* for withholding.

Definition 10.1 (Subsampled Cognitive Consensus)**.** Subsampled Cognitive Consensus (SCC) settles a Tier-2 cognitive task by: (1) drawing a committee of N providers at random from the eligible, reputation-weighted set using a deterministic beacon; (2) collecting commit-then-reveal signed responses over a finite output space; (3) declaring the task settled when at least a threshold θ of the committee reveals the same canonical answer, under one of three aggregation modes; and (4) emitting a PoT receipt and paying winners while slashing withholders.

10.1 Eligibility and the Sampling Draw

Sampling is sound only if the pool sampled from is honest in aggregate and the draw is unbiasable. SCC enforces both with an *eligible-set margin* and a *deterministic beacon*, exactly as realized in the `aivm` quorum chain.

The committee size is N with agreement threshold θ; the deployed defaults are N=5, θ=3 (a 3-of-5 quorum, with a hard floor $N \geq 3$). Before any draw, the eligible universe E—providers meeting the minimum bond and reputation—must exceed N by a margin:

$$|E| \;\geq\; N + \max\big(\,\rho_{\text{floor}},\; \lfloor N \cdot \rho_{\text{bps}}/10000 \rfloor\,\big), \qquad \rho_{\text{floor}} = 2,\; \rho_{\text{bps}} = 5000, \tag{2}$$

so the eligible set must be at least $N + \max(2, \lfloor N/2 \rfloor)$; for N=5 this is $|E| \geq 7$. The margin forbids the degenerate case where the eligible set is barely the committee, in which an adversary who controls the few eligible providers controls the answer. Building the eligible universe once and requiring it strictly larger than the draw is what gives the random selection room to land on honest providers.

The draw itself uses a deterministic beacon (a committed, unbiasable randomness source) so that committee selection is reproducible—every validator agrees who was sampled—yet unpredictable before the task is posted, so an adversary cannot position providers to be selected. Determinism of the draw is essential: it is part of what makes SCC's *settlement* deterministic (and hence Pattern-B-verifiable) even though the providers' generation is not.

10.2 Commit–Reveal: Independence Without Trust

If providers revealed answers in the open, the second provider could copy the first, collapsing the committee to a single point of view and defeating the redundancy that makes the quorum sound. SCC prevents this with a two-phase commit–reveal that is already implemented in the `aivm` chain, with two precise properties:

Sealed independence.
The reveal window opens *strictly after* the commit window closes (commit by height $\leq$ `CommitDeadline`;

Algorithm 1 Subsampled Cognitive Consensus: one task.

Input: task τ with question q, model m, output space $\mathcal{O}$, size N, threshold θ, escrowed fee; beacon ξ
Build eligible set E from R (bond $\geq$ `MinProviderBond`, reputation gate)
if $|E| < N + \max(\rho_{\text{floor}}, \lfloor N\rho_{\text{bps}}/10000 \rfloor)$ **then**
  abort: refund escrow (eligible set below margin)
end if
$C \leftarrow \textsc{WeightedSample}(E, N; \xi)$ (deterministic draw)
escrow $N\cdot$ rewardPerProvider; open *commit* window of `CommitBlocks`
for all provider $p \in C$ (off-chain, nondeterministic generation) **do**
  $o_p \leftarrow \textsc{Generate}_m(q) \in \mathcal{O}$; pick nonce r_p
  post operator-bound commit $h_p = H(p \,\|\, \text{outHash}(o_p) \,\|\, \text{embHash}(o_p) \,\|\, r_p)$
end for
close commit window; open *reveal* window of `RevealBlocks` (strictly after)
for all provider $p \in C$ that committed **do**
  reveal $(\text{outHash}(o_p), \text{embHash}(o_p), r_p)$; engine recomputes h_p, rejects mismatch
end for
aggregate revealed responses under mode κ (§10.4) $\rightarrow$ canonical o^*, distribution π
if $\geq \theta$ providers revealed o^* **then**
  emit PoT receipt; pay agreeing revealers (winners); **slash committed non-revealers**
else
  mark task *Failed*; refund requester; slash withholders
end if

reveal in `CommitDeadline` $<$ height $\leq$ `RevealDeadline`). No provider can observe a peer's revealed answer before its own commit is sealed, so each commit is an independent judgment. The deployed windows are `CommitBlocks` = `RevealBlocks` = 30 blocks.

Operator-bound commits.
The commit hash binds the committing provider's address: $h_p = H(p \,\|\, \text{outHash} \,\|\, \text{embHash} \,\|\, r_p)$. A provider who copies a peer's commit cannot reveal it—the recomputed operator-bound commit would not match—so commit-stealing is impossible, not merely discouraged.

These two properties together make the committee's responses genuinely independent samples, which is the statistical precondition for the agreement threshold to mean anything.

10.3 Slash Withholders, Not Dissenters

The most important economic rule in SCC is what it does *not* punish. A provider who commits and then fails to reveal (withholding) is slashed: withholding is an attack on liveness and is indistinguishable from a provider trying to see others' answers first. But a provider who reveals an honest answer that turns out to be in the minority—a *dissenter*—is **not** slashed. Dissent is information, not misbehavior (Section 16); punishing it would create a chilling effect in which providers echo the expected answer rather than report their genuine judgment, destroying the diversity the quorum depends on.

Principle 10.1 (Slash withholding, never dissent)**.** SCC slashes a committed provider that fails to reveal (a liveness fault), and slashes a provider that reveals a value inconsistent with its own

commit (equivocation). It never slashes a provider for revealing a minority answer. Honest dissent is recorded in π and paid no worse than its eligibility entitles; only withholding and equivocation forfeit bond.

The Sybil/forgery cost is bounded below by the bond: forging a canonical answer requires controlling at least θ revealing providers, each holding at least `MinProviderBond` ($1{\times}10^{18}$ wei per unit in the reference deployment), so the absolute cost floor of manufacturing a fraudulent receipt is $\theta{\cdot}$`MinProviderBond`, independent of any selection grinding—the eligible-set margin and the deterministic beacon remove the grinding, and the bond prices the residual.

10.4 Three Consensus Modes

A cognitive question is not always a question with one correct answer; sometimes it is a preference, sometimes an estimate. SCC therefore supports three aggregation modes κ, chosen per task, each appropriate to a different shape of question. These are detailed with their finiteness requirements in Section 11; here we name them and their settlement rule:

1. **Exact-hash quorum.** The answer is a single canonical value; settlement requires $\geq \theta$ providers to reveal the *same* `CanonicalOutputHash`. Used when the question has a determinate answer (a classification, a structured extraction, a yes/no). This is the mode the `aivm` quorum chain settles directly.

2. **Preference metastability.** The answer is one of a small finite ballot (e.g. `YES/NO/DELAY/UNSAFE`); settlement adopts the option that crosses the agreement threshold, with repeated subsampling driving the committee to a metastable majority exactly as in Avalanche, but over a cognitive ballot rather than over blocks. Used for judgment calls where "not yet / unsafe" is a legitimate, safety-preserving outcome.

3. **Market aggregation.** The answer is an estimate; the committee's responses are aggregated as a stake-weighted distribution (a prediction-market-style summary), and the receipt records the aggregate together with π. Used for quantitative forecasts where the wisdom of the weighted crowd, not a single point, is the product.

10.5 Finality

SCC finality has two stages, mirroring the two-step finality of the surrounding deterministic chains:

1. **Settlement finality** (one reveal window after commit): once $\geq \theta$ consistent reveals are recorded, the canonical answer is fixed and the PoT receipt is emitted on A. The answer cannot change thereafter without a fork of A.

2. **Verified finality** (Pattern B): the receipt is committed under a receipts root and verified on C (Theorem 8.1); only at this point does the answer have any effect on the canonical ledger. A consumer requiring ledger-level assurance waits for verified finality; a consumer needing only the cognitive conclusion may act on settlement finality at its own risk.

The metastability argument inherited from Avalanche gives SCC its safety/throughput profile: with an honest supermajority in the eligible set and an unbiasable draw, the probability that a sampled committee of N fails to reflect the eligible majority falls exponentially in N, so modest committee sizes (N=5, θ=3) yield strong agreement guarantees while keeping the per-task cost—N

paid generations—bounded. SCC is thus the cognitive analogue of Avalanche's repeated-subsampling consensus: same statistical engine, finite cognitive ballots in place of block preferences, and an economic layer (bond, escrow, slash-withholders) that pays for honest thought and prices forgery.

11 Finite Outputs: Prose as Evidence, Never Consensus

The single most important constraint on what a Thinking Chain may settle is that the settled answer must lie in a finite, declared output space. This constraint is what makes SCC's agreement threshold meaningful and what makes Pattern-B verification a decidable procedure. We state the rule, give the admissible output shapes, and explain how prose is handled without ever entering consensus.

Principle 11.1 (Finite outputs only)**.** A cognitive task admissible to settlement must declare, in advance, a finite output space $\mathcal{O}$, and its canonical answer o^* must satisfy $o^* \in \mathcal{O}$. Free-form prose is *never* the object of consensus. Where a thought naturally produces prose, the prose is stored off-chain and committed by hash as evidence accompanying the finite answer; consensus is reached on the finite answer and on the hash of the evidence, never on the prose itself.

11.1 Why Finiteness Is Non-Negotiable

Two honest large models asked an open-ended question produce different prose—different wording, different ordering, different emphasis—even when they agree in substance. If the settled object were the prose, no two providers would ever "agree," and the agreement threshold of SCC would be unreachable. Worse, verification would be undecidable: to check whether two prose answers "mean the same thing" is itself a cognitive judgment, which would require another cognitive task, which would require another, without termination. Finiteness cuts this regress: agreement on a finite answer is a hash comparison (exact-hash mode) or a count over a small ballot (preference mode) or a weighted sum (market mode)—each a deterministic, decidable operation. The reproducibility line of Section 9 and the finiteness line here are the two walls that together make cognition settleable.

11.2 Admissible Output Shapes

The three SCC modes correspond to three families of finite output space:

Categorical / structured (exact-hash mode).
$\mathcal{O}$ is an enumerated set, or the set of values of a fixed schema (a struct of bounded integer fields, an address, a class label, a JSON object normalized by a canonicalization scheme so that there is exactly one byte string per logical value). Agreement is equality of `CanonicalOutputHash`. Canonical JSON via RFC 8785 (JCS) [5] is the recommended normalization for structured answers: it removes whitespace and key-ordering ambiguity so that semantically identical objects hash identically, which is the precondition for two providers' structured answers to be recognized as the same answer.

Ballot (preference-metastability mode).
$\mathcal{O}$ is a small explicit set of options—e.g. $\{\texttt{YES}, \texttt{NO}, \texttt{DELAY}, \texttt{UNSAFE}\}$. Agreement is the option crossing the threshold. The presence of `DELAY`/`UNSAFE` as first-class options is deliberate: it lets the network conclude "do not act yet" as a settled, safety-preserving answer rather than being forced into a binary.

Bounded numeric (market-aggregation mode).
$\mathcal{O}$ is a bounded numeric range with a declared quantization (e.g. a risk score in $\{0, \ldots, 100\}$, a probability quantized to basis points). Agreement is a stake-weighted aggregate within the range; the receipt records the aggregate and the distribution π.

11.3 The Role of Prose

Prose is not banned; it is demoted from *decision* to *evidence*. A Tier-2 provider answering "rate the risk of this contract" produces both a finite answer (a bucket in $\{0, \ldots, 100\}$) and a prose rationale. The finite answer is what SCC settles and what the PoT receipt commits; the rationale is stored off-chain (e.g. in content-addressed storage on M/D) and its hash is the `embHash` committed in the commit–reveal (Algorithm 1). This yields the best of both: the network reaches deterministic agreement on the decision, while preserving a tamper-evident, retrievable explanation for audit, dispute, and human review. The rationale is excluded from the consensus preimage precisely so that a difference in wording cannot block agreement on substance—a discipline we apply verbatim to governance in Section 15, where the vote preimage is {vote, confidence_bucket} with the free-text rationale deliberately excluded.

Principle 11.2 (Inspectable without an LLM)**.** Because every settled object is finite and every receipt is a fixed-width record, the entire cognitive history of the network is auditable by a verifier that runs *no model at all*. One can check what was asked, what was concluded, by whom, with what dissent, and that each conclusion was legitimately settled and verified, using only hashing and signature checks. The prose evidence is available for a human or a model that wishes to read it, but the integrity of the network's cognition does not depend on anyone re-running an LLM.

Principle 11.2 is a direct corollary of finiteness and is one of the strongest properties of the architecture: a Thinking Chain is not a black box that emits opaque AI verdicts. It is a glass box whose every conclusion is a finite, signed, proven fact, with the reasoning attached as readable but non-binding evidence.

12 Recursive Thinking and Budgets

Genuine deliberation is recursive: answering a hard question well often requires decomposing it into subquestions, answering each, and composing the results. Because PoT receipts compose (Section 4), a Thinking Chain can express this directly—a cognitive task may spawn child tasks whose receipts are evidence for the parent. Recursion is the source of the network's depth of thought and, unmanaged, the source of its most dangerous failure: a network that can spend money to think can think itself into an unbounded spend. Bounded recursion is therefore a constitutional requirement (Section 16), and this section gives the mechanism that enforces it.

12.1 The Two Budgets

Every cognitive task carries two non-replenishable counters, inherited and strictly decreased by its children:

Definition 12.1 (Cognitive budget)**.** A task τ carries a *depth budget* $d(\tau) \in \mathbb{N}$ and a *token/fee budget* $b(\tau) \in \mathbb{N}$. A child τ' spawned by τ must satisfy $d(\tau') \leq d(\tau) - 1$ and $\sum_{\tau' \in \text{children}(\tau)} b(\tau') \leq b(\tau) - c(\tau)$, where $c(\tau)$ is the fee already committed by τ itself. A task with $d(\tau) = 0$ may spawn no children; a task whose remaining fee budget cannot cover a child's escrow may not spawn it.

The depth budget bounds the height of the recursion tree; the fee budget bounds its total cost. Both are checked in the deterministic settlement path (when a child intent is emitted via Pattern A, its inherited budgets are validated against the parent's committed remaining budget), so the bound is enforced by the state-transition function, not by the good behavior of the cognitive layer. This is the attention organ of Definition 3.1 made concrete: the budgets are how the network decides how hard to think and when to stop.

12.2 Termination

Proposition 12.1 (Recursion terminates within budget)**.** *Under Definition 12.1, the recursion tree rooted at a task τ_0 with depth budget d_0 and fee budget b_0 has height at most d_0 and total committed fee at most b_0; in particular the tree is finite and the total spend is bounded by b_0 regardless of the cognitive layer's behavior.*

Proof. Height: along any root-to-leaf path the depth budget strictly decreases by at least 1 per level (Definition 12.1) and is a natural number bounded below by 0, so any path has length at most d_0; hence the tree has height at most d_0 and, being finitely branching with bounded height, is finite. Fee: by induction on the tree, the total fee committed in the subtree rooted at τ is at most $b(\tau)$—the base case is a leaf committing $c(\tau) \leq b(\tau)$, and the inductive step sums the children's subtree totals $\sum b(\tau') \leq b(\tau) - c(\tau)$ with τ's own $c(\tau)$, giving $\leq b(\tau)$. At the root this is b_0. Both bounds are enforced in the deterministic path, so they hold irrespective of λ (Theorem 8.1). □ □

Proposition 12.1 is what makes recursive cognition safe to offer at all. Without it, the obvious attack is trivial: pose a question whose "best answer" is to ask a slightly different question, and watch the network spend its treasury exploring an infinite regress. With it, the worst case is bounded a priori by the budgets the requester escrowed, and an attacker who wants the network to spend more must escrow more themselves.

12.3 Recursion as Attention Allocation

The budgets do more than prevent runaway spend; they are the network's mechanism for allocating attention. A high-stakes question (escrowed with a large fee budget and generous depth) can decompose into a deep tree of careful sub-deliberations; a routine question (small budget, depth zero) is answered in one shot or routed to Tier 1. The scheduler that orders tasks on A under contention—preferring high-salience, high-stake tasks when cognitive load u is high—is the network deciding what to think about, and the budgets are the network deciding how hard. Together they realize attention as a first-class, accountable, bounded resource, which is precisely the organ a ledger lacks (Section 3).

13 Chain-to-Chain Payments

A cognitive economy needs settlement rails as much as it needs a consensus mechanism. The same Bridge Law that governs thought governs money: value crosses between organs only as a committed, verifiable artifact, never as a live promise. We give four payment patterns, ordered from the simplest to the most expressive, each appropriate to a different cadence of cognitive work. All four respect Corollary 8.2: a stalled cognitive layer can delay a payment but can never create or destroy value.

13.1 Pattern P1: Escrow-Settle

The default, used by every `TaskRequest` (Section 6). The requester escrows the maximum fee on C *before* the task is posted; the escrow is released by Pattern-B verification of the resulting PoT receipt. No provider thinks for an unfunded promise, and no requester pays for an unsettled answer. Escrow-settle is atomic with respect to the cognitive outcome: if the task reaches quorum, winners are paid and the model/data royalties flow; if it fails, the requester is refunded; in neither case can a partial or duplicate payment occur, because the release is gated on a once-verifiable receipt with replay protection.

13.2 Pattern P2: Netting

When two organs exchange a high volume of cognitive work in both directions (e.g. C continually asks A to classify, while A continually asks D for provenance), settling each task individually is wasteful. Netting batches a window of mutual obligations and settles only the difference. The set of receipts in the window is committed as a Merkle root; a single net transfer settles the balance; each individual receipt remains independently verifiable for audit. Netting reduces on-chain settlement traffic by an order of magnitude in steady state without weakening per-receipt verifiability—the net is an optimization over P1, not a replacement for its guarantees.

13.3 Pattern P3: Streaming

Some cognitive services are continuous rather than discrete: an availability commitment to keep a model loaded and answer within a latency bound, or a standing subscription to a sensory feed on S. These are paid by a stream—a per-block drip from a funded channel, claimable by the provider against committed heartbeat or uptime proofs. Streaming pays for *readiness to think*, complementing P1's payment for *thoughts produced*; it is the mechanism behind the availability reward of the first prototype (Section 18). A stream is closed by either party at any time, with the unspent balance returning to the funder, so no party is locked into a service that has stopped performing.

13.4 Pattern P4: Lock-Mint (Cross-Organ Value)

Moving the native value of one organ into another—paying an A provider in C's token, or moving a fee from G to D—uses lock-mint, the standard trustless bridge construction: value is locked on the source chain under a committed record, and a matching amount is minted on the destination only against a verified proof of the lock; the reverse burns and unlocks. Lock-mint inherits its security from the cross-chain proof system, requiring a supermajority attestation over the locking event, with mint/burn accounting that must balance and a challenge window for large transfers. It is the value analogue of Pattern B: the destination chain never trusts that a lock occurred, it *verifies* the committed proof, so the same liveness/safety decoupling (Corollary 8.2) applies—a bridge stall delays cross-organ payment but cannot mint unbacked value.

13.5 Choosing a Pattern

Table 4 summarizes. The selection is by cadence: one-shot thoughts use P1; dense bidirectional traffic uses P2 over P1; continuous readiness uses P3; cross-token movement uses P4. All four are compositions of the same two primitives—commit a record, verify it before acting—which is the Bridge Law applied to money. There is exactly one way to move value between organs (verify a committed proof) and these patterns are its specializations, not alternatives to it.

Table 4: Chain-to-chain payment patterns.

Pattern	Cadence	Pays for	Settlement trigger
P1 Escrow-settle	per task	a settled thought	Pattern-B receipt verify
P2 Netting	per window	batched mutual work	net of committed receipts
P3 Streaming	per block	readiness to think	heartbeat/uptime proof
P4 Lock-mint	on demand	cross-organ value	verified lock proof

14 Reputation and Eligibility

Subsampled Cognitive Consensus is only as sound as the pool it samples from. If the eligible set is dominated by a single model, a single operator, or a clique of colluders, the agreement threshold certifies conformity rather than truth. The reputation organ R (Section 5) governs who may be sampled and with what weight, and it is engineered for one goal above all: a sampled committee that is both *competent* and *diverse*.

14.1 Sampling Weight

Each provider p carries a reputation $\mathrm{rep}(p) \in [0, 1]$ derived from its settled history: the fraction of tasks in which it revealed (did not withhold), the fraction in which its honest answer agreed with the settled canonical answer, and the freshness-weighted recency of that record. Its sampling weight in the draw of Algorithm 1 is

$$w(p) \;=\; \mathrm{bond}(p)^{1/2} \cdot \mathrm{rep}(p) \cdot \mathrm{div}(p), \tag{3}$$

where $\mathrm{bond}(p)$ is the provider's bonded stake (entering sub-linearly, so capital alone cannot monopolize selection), $\mathrm{rep}(p)$ is the reputation above, and $\mathrm{div}(p)$ is the diversity factor below. The sub-linear bond exponent is deliberate: it makes stake necessary (it is the Sybil cost and the slashing collateral, Section 10) without making the committee an oligarchy of the richest providers.

14.2 Diversity: The Antidote to Monoculture

The deepest risk to a cognitive quorum is correlation. Five providers running the *same* model weights are not five independent samples; they are one sample with five signatures, and they will agree by construction even when they are wrong (Section **??**). The diversity factor $\mathrm{div}(p)$ counteracts this by reducing the marginal sampling weight of providers that are correlated—in model lineage, in operator, in geography, in infrastructure—with providers already likely to be drawn. Concretely, R maintains, per provider, a profile of (model family, operator identity, hosting region/provider), and the draw penalizes adding a member whose profile duplicates one already selected, so the committee tends toward independent vantage points. The `ModelSpec` registry (Section 18) makes model lineage explicit and therefore makes lineage-diversity enforceable rather than aspirational.

Principle 14.1 (Diversity is a consensus parameter, not a nicety)**.** The statistical validity of an SCC agreement threshold depends on the sampled responses being approximately independent. Diversity-weighted, correlation-penalized sampling is therefore not a fairness feature but a soundness requirement: without it, the quorum measures agreement among copies, and the threshold certifies monoculture rather than truth. A Thinking Chain must enforce committee diversity in lineage, operator, and infrastructure as a first-class consensus parameter.

14.3 Eligibility Gates

To enter the eligible set E of Equation 2, a provider must clear hard gates that are checked deterministically before the draw: a minimum bond at least `MinProviderBond`; a non-zero reputation (a provider with a record of withholding or equivocation falls out of eligibility); registration of the `ModelSpec` it serves in the governance-approved model set (Section 16); and a diversity-floor check that prevents the eligible set itself from collapsing to a monoculture. These gates, combined with the eligible-set margin, are what guarantee that the random draw has honest, independent providers to land on.

14.4 Reputation Dynamics

Reputation rises slowly and falls quickly, the standard asymmetry for trust under adversarial pressure. Honest agreement increments reputation with diminishing returns; withholding and equivocation decrement it sharply (in addition to the bond slash of Section 10); honest dissent is *neutral* to reputation (Principle 10.1)—a provider that consistently reports a genuine minority view is not penalized in standing, because its dissent is exactly the independent signal the quorum exists to capture. This dynamic, settled on R and fed into the next draw, closes the loop: the network learns which providers think well, samples them more, and continually refreshes the diversity of the pool so that competence never hardens into monoculture.

15 Governance: AI-Assisted, Not AI-Ruled

A network that can think will be asked to govern itself. The central design question is not whether AI participates in governance—it should, and powerfully—but where the line falls between AI *assisting* a decision and AI *making* it. We place that line explicitly and enforce it in the deterministic path. The governing principle is that cognition may inform, draft, analyze, and recommend, but the act of binding the network is gated by humans and time.

Principle 15.1 (AI-assisted, not AI-ruled)**.** The cognitive layer may produce governance *recommendations*—as PoT receipts over a finite ballot (e.g. `APPROVE/REJECT/NEEDS-REVISION`)—and these recommendations may be authoritative inputs to a decision. But every binding governance action passes through a deterministic, human-gated, timelocked path that no quantity of AI deliberation can bypass. The network may recommend any change to itself; it may enact, unilaterally, only the changes the constitution already permits it to enact (Section 16).

15.1 The Governance Vote Preimage

Governance votes are settled as cognitive receipts, and therefore obey finite-output discipline (Section 11). The consensus preimage of a governance vote is exactly $\{\text{vote}, \text{confidence\_bucket}\}$—the finite ballot choice and a quantized confidence—with the free-text rationale *excluded* from the preimage and committed only by hash as evidence. This is the same discipline applied to provider rationales in Section 11, and it is realized in the operator/canonical governance-consensus code. The exclusion is load-bearing: if rationale text were in the preimage, two voters who agree in substance but differ in wording would fail to form a quorum, and governance would deadlock on prose. By settling on $\{\text{vote}, \text{confidence\_bucket}\}$ and preserving the rationale as inspectable evidence, the network reaches deterministic agreement on *what* was decided while retaining a readable record of *why* each participant decided it.

15.2 The Human-in-the-Loop Hierarchy

Different governance actions warrant different degrees of human control. We define six levels, from fully autonomous to fully manual, so that each action can be assigned the minimum human burden consistent with its risk. Level assignment is itself a constitutional parameter (Section 16) and can only be *raised* (more human control) by the cognitive layer, never lowered.

Table 5: Human-in-the-loop levels. Each governance or agentic action is assigned a level; the cognitive layer may raise an action's level but never lower it.

Level	Name	Human role
0	Autonomous	None. AI acts on its settled receipt within pre-authorized bounds (e.g. route a Tier-1 classification, pay a quorum).
1	Notified	AI acts; humans are notified and may audit after the fact.
2	Vetoable	AI acts after a delay during which any authorized human may veto (timelock as circuit-breaker).
3	Approved	AI proposes; a human (or quorum of humans) must approve before the action fires.
4	Co-decided	AI recommends with confidence; a human decides, with the AI recommendation as a binding input of record.
5	Manual	AI may analyze and advise but may not propose a binding action; a human originates and enacts.

Routine, low-stakes, reversible actions sit at Level 0–1 (the network must be allowed to act on its own conclusions, or it is not an agent at all). High-stakes or irreversible actions—treasury disbursements above a threshold, changes to quorum parameters, additions to the eligible model set—sit at Level 3–4. Anything touching the constitution or the authority structure itself sits at Level 5 and additionally requires the constitutional amendment path (Section 16). The hierarchy is a dial, not a switch: it lets a network grant its cognition broad autonomy in the safe regions of its action space while reserving human judgment for the dangerous ones.

15.3 Conviction and Dissent in Governance

Governance aggregation preserves the same properties as cognitive aggregation. Voting power may scale with conviction (stake duration), as in the first prototype's conviction-voting model (Section 18), to weight committed long-term participants over transient capital. And the dissent distribution is preserved on-chain: a proposal that passed $51-49$ is recorded as contested, and the constitution may require a higher human-loop level or a longer timelock for narrowly-decided actions than for broadly-supported ones. The network remembers not just what it decided but how divided it was, and it can be made more cautious precisely when it is most divided.

16 The Constitutional Layer

The constitution is the set of rules a Thinking Chain cannot think its way past. It is the seventh organ of Definition 3.1, the one with no cognitive part (Table 1): pure deterministic predicate, enforced in the state-transition function, amendable only through a path the cognitive layer cannot unilaterally walk. This section enumerates the constitutional rules, states the meta-rule that

protects them, and explains why each is necessary. These are not policy preferences; they are the invariants without which the rest of the architecture is unsafe.

16.1 The Twelve Rules

1. **Deterministic consensus.** The canonical state transition is deterministic; no consensus-relevant value may be nondeterministic. (Foundation of Theorem 8.1.)
2. **No live thought in state transition.** A state transition may not depend on live thought or a live cross-chain query; cognition enters only via Pattern A / Pattern B. (The Bridge Law, Principle 8.1.)
3. **Committed proofs for cross-chain effects.** Any effect of one organ on another's state requires a committed, verifiable proof or receipt; no organ trusts a message by its arrival. (Principle 5.1.)
4. **Structured outputs only.** A settleable cognitive answer must lie in a finite, declared output space; prose is evidence, never consensus. (Principle 11.1.)
5. **Bounded recursion.** Every cognitive task carries a depth budget and a fee budget that strictly decrease across children; recursion provably terminates within budget. (Definition 12.1, Proposition 12.1.)
6. **Replay-protected receipts.** Each PoT receipt may be verified-and-acted-upon at most once; a verified intent is retired on use. (Pattern B, Section 8.)
7. **Authority expansion is human-gated and timelocked.** Any change that enlarges the set of actions the cognitive layer may take without human approval requires a human/DAO signature *and* a timelock. (The meta-rule, below.)
8. **ModelSpec-registered governance models.** Only models whose `ModelSpec` is registered and governance-approved may answer governance questions or be admitted to a governance-relevant quorum; the eligible model set is itself a governed parameter.
9. **Preserve dissent and confidence.** The full confidence/dissent distribution π of every settled question is recorded; the network may not collapse a contested conclusion into a unanimous one, and may not slash honest dissent. (Principle 10.1.)
10. **Inspectable without an llm.** The integrity of the network's cognition must be checkable using only hashing and signature verification, with no model execution required. (Principle 11.2.)
11. **Slash withholding, not dissent.** Economic penalties attach to liveness faults (withholding) and equivocation, never to honest minority answers. (Principle 10.1.)
12. **No self-granted authority.** The cognitive layer may *propose* an expansion of its own authority but may not *enact* one; enactment requires the human-gated, timelocked path of Rule 7. (The meta-rule, below.)

16.2 The Meta-Rule

Rules 7 and 12 are the constitution's self-protection, and they are worth stating as a single principle because together they are what prevents a cognitive network from bootstrapping itself out of every other constraint.

Principle 16.1 (AI may propose more authority, but may not grant itself more authority)**.** There is an asymmetry, enforced in the deterministic state-transition function, between proposing and enacting an expansion of cognitive authority. The cognitive layer may originate, deliberate on, and recommend—as finite governance receipts—any change to its own powers, including the constitution itself. But the predicate that enacts such a change requires a human/DAO signature and a timelock, and that predicate is not reachable by any sequence of purely cognitive actions. Consequently the network can argue for more power, persuasively and with evidence, but the lever that grants it is, by construction, outside the network's own reach.

Why this asymmetry and not, say, a high cognitive quorum? Because any threshold the cognitive layer can satisfy on its own is a threshold it can be argued or manipulated into satisfying; the only robust barrier to self-amplification is one the cognitive layer *cannot* satisfy alone, no matter how much it deliberates. A human signature is exactly such a barrier: it is an input from outside the cognitive system. The timelock adds a second guarantee—that even a compromised human gate cannot enact instantly, leaving a window in which other humans can observe and intervene. The pair (external signature + delay) is the minimal construction that makes "the network cannot grant itself power" a theorem about the state machine rather than a hope about its operators.

16.3 Amending the Constitution

The constitution is not immutable—an immutable constitution cannot adapt to genuinely new circumstances—but it is the hardest thing in the system to change. A constitutional amendment is a Level-5 governance action (Section 15) that additionally requires: a supermajority of human/DAO signatures, the longest timelock in the system, and a published amendment receipt that records the prior and proposed text by hash so the change is auditable forever. The cognitive layer may draft and advocate an amendment; it cannot ratify one. This is the constitution applying its own meta-rule to itself: the network may propose to change its limits, but only its human principals may enact the change, and only slowly, in the open.

16.4 Why a Constitution at All

A reader may ask why a deterministic, Bridge-Law-respecting network needs a constitution beyond the Bridge Law itself. The answer is that the Bridge Law makes the network *safe to its own ledger*—it cannot corrupt its state—but says nothing about whether the network's *actions*, all individually deterministic and well-formed, might in aggregate be undesirable: spending its treasury unwisely, concentrating authority, acting irreversibly on a contested conclusion, or amplifying its own powers. The constitution constrains the *policy* of a network that is already safe in its *mechanism*. Theorem 8.1 guarantees the network cannot break; the constitution guarantees that an unbroken network stays within bounds its principals chose. Both are required, and they are orthogonal: the first is about determinism, the second about authority.

17 Node Cognitive Architecture

The determinism partition (Section 1) must be realized not only in the protocol but in the software a validator runs. A node that loaded a large language model into the same process as its consensus engine would have one careless code path away from letting nondeterministic model output influence a state root. We therefore prescribe a node architecture in which the cognitive machinery is physically separated from the consensus machinery by a process boundary and a firewall that only signed, structured artifacts may cross.

17.1 The Sidecar Pattern

A Thinking Chain validator is two processes, not one:

The consensus node.
The deterministic core: it runs the EVM (including the Tier-1 inference precompile, which is pure integer code in-process and therefore safe), validates and produces blocks, maintains state, and verifies receipts via Pattern B. It contains *no* large model and makes *no* network call to a model during block execution. Its execution is a pure function of committed inputs (Theorem 8.1).

The cognitive sidecar.
A separate process (or pool of processes) that hosts Tier-2 models (the HANZO engine, Section 9), performs off-chain generation when this node is a sampled provider, computes commit hashes, and reveals responses. The sidecar may be GPU-bound, nondeterministic, and as heavy as the operator likes; it never touches consensus state directly.

The two communicate only through the cognitive economy's message types (Section 6): the sidecar learns of tasks it was sampled for by reading committed A state, and it influences state only by submitting commit/reveal transactions that are settled by SCC and verified like any other. The process boundary makes the partition a deployment fact, not merely a coding convention: even a buggy sidecar cannot inject a value into the consensus node's state root, because the only channel between them is the same committed-transaction channel any external party uses.

17.2 The Signer Firewall

The most sensitive component is the validator's signing key, which attests to blocks and (if the node is a provider) to cognitive responses. The *signer firewall* is a strict rule about what that key will sign:

Principle 17.1 (Signer firewall)**.** A validator's signer signs only (i) canonical block attestations over deterministic state, and (ii) canonical, finite cognitive artifacts (commits and reveals over a declared output space). It never signs free-form model output, never signs a value it has not type-checked against a declared schema, and never signs on behalf of the cognitive sidecar without the sidecar's response having passed the finite-output and commit-binding checks. The firewall sits between the sidecar and the key: model output reaches the key only after being reduced to a finite, schema-valid, operator-bound artifact.

The firewall is the local enforcement of the global finite-output rule (Principle 11.1). Its purpose is to ensure that even a compromised or hallucinating sidecar cannot get the validator's key to sign something outside the finite output space—the worst a bad sidecar can do is cause its own commit to be wrong (and thus its own reveal to fail, slashing only itself), never to forge a signature over arbitrary content. Combined with operator-bound commits (Section 10), the firewall ensures a node is accountable for exactly what it signed and nothing more.

17.3 Node Topology

Figure 3 shows the validator. The consensus node and sidecar are separate failure domains: the sidecar can crash, be upgraded, swap models, or run on entirely separate hardware (even a separate machine, or the federated multi-vendor backends used for training) without affecting the consensus node's ability to validate and produce blocks. A validator that runs no sidecar at all is still a complete consensus participant—it simply earns no Tier-2 provider rewards. This graceful degradation

is a direct consequence of the partition: cognition is an opt-in earning role layered on top of a consensus node, not a requirement of running one.

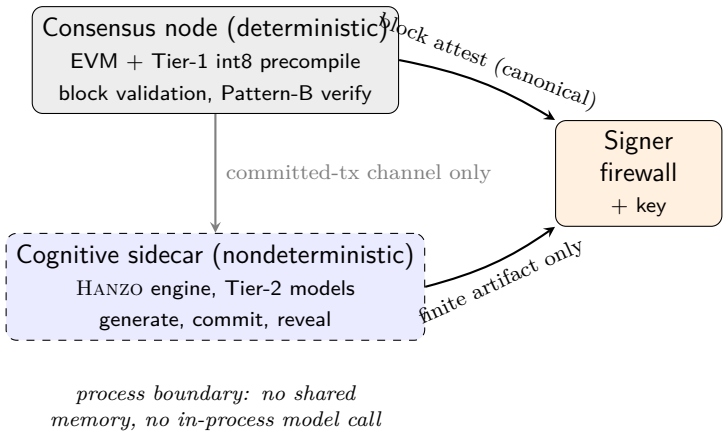

Figure 3: Validator topology. The consensus node (deterministic, holds the Tier-1 integer precompile in-process) and the cognitive sidecar (nondeterministic, holds Tier-2 models) are separate failure domains communicating only over the committed-transaction channel. The signer firewall (Principle 17.1) admits only canonical block attestations and finite, schema-valid cognitive artifacts to the signing key.

18 Beluga L3: The First Prototype

The architecture of this paper is general, but it is not hypothetical. Its first instantiation is Beluga, the Zoo Layer-3 application chain for AI model economics, specified in full in the companion Beluga whitepaper [10]. Beluga is a Thinking Chain in the precise sense of Definition 3.1: a deterministic EVM chain (the C role), surrounded by a verifiable cognitive layer, settling AI economic decisions as receipts. This section maps Beluga's concrete mechanisms onto the general architecture and points to the running code, so that the reader can see which parts of this paper are already built. We do not re-derive Beluga's token economics, contract suite, or consensus adaptation—those are the subject of [10]; we show only how Beluga realizes the Thinking Chain pattern, and *to what end*.

18.1 Beluga's Telos: Conservation, Community, and Self-Funding Cognition

A Thinking Chain is a mechanism; Beluga gives the mechanism a *purpose*. Beluga is named for the beluga whale, and it exists not to demonstrate cognitive consensus in the abstract but to direct cognitive surplus toward a real-world public good: ocean conservation. This is the thesis the first prototype is built to make concrete—that a conscious blockchain can have a *conscience*, in the operational sense that its cognitive economy is consciously aimed. Three commitments define this telos.

Awareness. Beluga is deliberately approachable—a small, friendly model with a whale as its mascot and public face—because for a conservation project, attention is itself an instrument. Every interaction with the chain's model is an interaction with the cause: the model is the campaign, and the campaign is self-explaining because the chain can answer questions about its own mission from its own state (Definition 3.1, perception).

Funding. BELUGA routes a fixed *conservation tithe* from every POT settlement into a conservation treasury governed by the ZOO Labs Foundation, a 501(c)(3). The cognitive economy of Section 6—the fees that requesters, providers, and governance pay to settle thought—thereby becomes a renewable funding stream for ocean and beluga-whale conservation. The tithe is a protocol parameter under the constitution (Section 16): it can be changed only through the G human-loop at Level 4–5 (Section 15), so the cause cannot be quietly defunded by an automated proposal, and the autonomy ceiling forbids the cognitive layer from redirecting the tithe to itself.

A self-funding, community-upgraded model. BELUGA's brain is community-owned. Its deterministic Tier-1 model (the `zen-nano` port of Section 9) and its Tier-2 large models are improved by community contributions through *DeltaSoup* aggregation over the *Experience Ledger* provenance graph [11]: contributors submit fine-tuning deltas, the ledger (the D provenance organ) records who contributed what, and the `Invoice` royalty split of Section 6 pays contributors out of inference fees. The model is therefore *mutually upgraded*—it improves the more the community uses and trains it—and *self-funding*: the same inference fees that pay for upgrades also pay the operators who serve it and the conservation tithe. No central party underwrites the model; the community that benefits from it funds it, and a constitutionally-protected share of that flow funds the whales. BELUGA is thus the existence proof for a broader claim of this paper—that a Thinking Chain's verifiable cognitive economy can be pointed, by construction and under human-governed law, at a public good rather than only at extraction.

18.2 Beluga as an Instantiation of the Organs

BELUGA maps onto the organ model (Section 5) as follows. Its EVM is the C cortex: model registration, inference requests, and bounty escrow are deterministic transactions, and its inference-pricing precompile at `0x0300` computes fees deterministically in-consensus. Its inference marketplace and provider network are the A cognition organ: large-model inference is served off-chain and settled by quorum. Its data-contribution layer (*DeltaSoup*, grounded in the Experience Ledger [11]) is the D provenance organ, carrying the royalty graph that the `Invoice` message (Section 6) splits payment across. Its evaluation-and-red-team market is a specialized cognitive task type. Its `BelugaGovernor` with conviction voting is the G self-governance organ, and its model registry is the `ModelSpec` substrate that Rule 8 of the constitution (Section 16) governs. BELUGA thus exercises perception (pricing/observation), memory (the registry and ledger), agency (contracts acting on settled results), and self-governance—the cognitive organs of Definition 3.1—within an inherited, economically-secured validator set.

18.3 The Realized Mechanisms

The load-bearing mechanisms of this paper are implemented in the LUX and HANZO codebases that BELUGA builds on. We list them with their addresses and roles so the grounding is explicit:

The `aivmbridge` precompile implements Pattern A (`submitInferenceIntent`, selector `0x10000000`) and Pattern B (`verifyInferenceReceipt`, selector `0x11000000`) exactly as in Section 8, with the deterministic intent-id construction and the keccak–Merkle receipt proof. The `aivm` chain implements SCC's eligible-set margin ($E \geq N + \max(2, N/2)$), the deterministic beacon draw, operator-bound commit–reveal with strictly-ordered windows (30/30 blocks), and slash-withholders-not-dissenters, with the `AInferenceReceipt` record of Listing 1. The deterministic inference precompile implements Tier-1 as pure-Go `CGO=0` int8 transformer code (Listing 5). The operator/canonical library implements the governance vote preimage $\{\text{vote}, \text{confidence\_bucket}\}$ with rationale excluded (Section 15).

Table 6: Realized components of the Thinking Chain architecture, in the Lux and Hanzo codebases. These exist, compile, and are tested; Beluga composes them.

Component	Address / module	Role in this paper
Deterministic int8 inference	`precompile/inference`, `0x0300...03`	Tier-1, §9
Cognitive bridge (C→A)	`precompile/aivmbridge`, `0x0300...04`	Bridge Law, §8
Quorum-settlement VM	`chains/aivm`	scc + receipts, §§4,10
Model registry	`precompile/modelregistry`	`ModelSpec`, §§14,16
Tier-2 provider runtime	Hanzo `engine` / `engine-ffi`	Tier-2, §9
Commit–reveal + canonical lib	`aivm`, operator/canonical	§§10,15

18.4 The Empirical Result

The architecture's central physical claim—that small integer models are bit-reproducible across heterogeneous hardware while large floating-point models are not (Principle 9.1)—was tested directly in this stack. The same engine, given the same input and committed weights, produced an *identical output hash* across two independently implemented EVMs (the Go EVM and the Rust EVM). This cross-VM bit-identity is the empirical justification for running Tier-1 in consensus: two independently written virtual machines agreeing byte-for-byte is strong evidence that the forward pass is genuinely deterministic. The corresponding negative observation—that large models do not reproduce across hardware, because floating-point reductions are not associative—is why Beluga's large-model inference is settled by quorum (Tier-2) rather than recomputed. The two-tier split is therefore not a design preference but a measured consequence of how integer and floating-point arithmetic behave across machines.

18.5 What Beluga Proves and What Remains

Beluga proves that the Thinking Chain pattern is buildable on today's infrastructure: the bridge, the deterministic inference, the quorum settlement, the registry, and the provider runtime are real, and they compose into a chain that pays for settled AI economic decisions. What Beluga does not yet do is exercise the *full* organism at scale—the complete seven-organ decomposition, deep recursive deliberation, and the full constitutional and human-loop machinery are specified here and partially realized, not yet deployed end to end. Beluga is the first prototype Thinking Chain; the general architecture of this paper is the blueprint for the class it inaugurates. The remaining sections analyze what must hold for that class to be safe at scale.

19 Economic Roles and Slashing

A Thinking Chain is an economy, and its safety rests as much on incentives as on cryptography. This section names the five economic roles, what each is paid for, and what each is slashed for. The unifying rule is the one established in Section 10: pay for honest, settled cognitive work; slash liveness faults and equivocation; never slash honest dissent.

19.1 The Five Roles

Providers.
The agents who think. They host models (Tier-2) and answer sampled cognitive tasks via commit–reveal. Paid per agreeing reveal (Equation 1) and by availability streams (Pattern P3) for keeping

models loaded. Bonded; slashed for withholding and equivocation. Providers are the supply side of thought.

Validators.
The agents who agree. They produce and attest blocks on the deterministic chains and verify receipts via Pattern B. Paid block rewards and fees. Bonded; slashed for double-signing and consensus faults. A validator may additionally run a sidecar and thereby also be a provider, but the roles are distinct and separately bonded (Section 17).

Agents.
The principals who pose questions and act on answers—autonomous on-chain agents and the humans behind them. They escrow fees (Pattern P1) to demand cognition and consume the resulting receipts. Agents are the demand side; their escrow funds the entire economy.

Red teams.
The agents who attack the network's cognition for profit. They are paid bounties for discovering inputs that cause incorrect, inconsistent, or unsafe settled answers—adversarial prompts, monoculture-exploiting questions, recursion bombs caught before they fire. Red teams are a *first-class economic role*, not an afterthought: a network that pays for the discovery of its own cognitive failures hardens continuously, and their findings are recorded as evidence on M for future deliberations to consult.

Memory curators.
The agents who keep the network's memory useful. They curate, deduplicate, and index the settled PoT receipts and the data on D/M, and are paid by the downstream value of the memory they maintain (royalty when a curated dataset trains a model that later earns, per the `Invoice` split of Section 6). Curators are why memory is an asset rather than a landfill.

19.2 The Slashing Table

Table 7 states the slashing conditions across all bonded roles. Two design rules govern it. First, every slash attaches to an action that is *deterministically detectable from committed state*—a missing reveal after a committed commit, a reveal inconsistent with its commit, a duplicate signature—so that slashing itself never requires a cognitive judgment (it would be incoherent to need a quorum to decide whether to slash a quorum member). Second, no row punishes a minority answer: dissent is information (Principle 10.1, constitutional Rule 9).

Table 7: Slashing conditions. Every condition is deterministically detectable from committed state; honest dissent appears nowhere.

Role	Violation	Penalty
Provider	Committed but did not reveal (withholding)	bond slash
Provider	Reveal inconsistent with commit (equivocation)	full bond
Provider	Serving an unregistered `ModelSpec`	eligibility loss
Validator	Double-signing a block height	full bond
Validator	Attesting an invalid state transition	full bond
Provider	Revealing a *minority* honest answer	**none**

19.3 The Forgery Floor

The economic security of a single PoT receipt is bounded below by the cost of manufacturing a fraudulent one. To forge a canonical answer, an adversary must control at least the threshold θ of revealing providers in a sampled committee. The eligible-set margin (Equation 2) and the deterministic beacon together remove selection grinding—the adversary cannot cheaply arrange to be sampled—so the residual cost is the bond the adversary must put at risk: at least $\theta \cdot$ `MinProviderBond`, forfeited when the forgery is detected (by a dissenting honest provider, a red team, or a verifier). With the reference θ=3 and `MinProviderBond` = 1×10^{18} wei per unit, the floor is 3 units of bonded value per forged receipt, scaling with the deployment's bond denomination and with N, θ. The network operator sets these parameters to price forgery above the value any single receipt controls, and high-value questions can demand larger N, θ to raise the floor accordingly. Economic security and committee size are thus a single tunable: thinking harder (larger N) and pricing forgery higher (larger $\theta\cdot$ bond) are the same dial.

20 Failure Modes and Mitigations

A Thinking Chain inherits every failure mode of a classical chain and adds new ones at the cognitive seam. The architecture is only as strong as its response to these. We enumerate the failure modes that are specific to verifiable cognition—each with its mechanism, the property it threatens, and the structural mitigation already present in the design—and mark which are closed by construction versus which remain economic or social risks that policy must manage.

20.1 Consensus Leakage

Mechanism. A live model output, or a live query to another chain's mutable state, influences a deterministic state transition; two validators executing the same block at different instants observe different cognition and compute divergent state roots.

Threatens. Safety—the chain forks.

Mitigation (closed by construction). The Bridge Law (Section 8): a deterministic chain may only emit an intent (Pattern A) or verify a committed receipt with proof (Pattern B). Theorem 8.1 establishes that under the Bridge Law the state root is a pure function of committed inputs, independent of the cognitive layer's liveness. Tier-1 inference is admitted into consensus only because it is bit-reproducible (Principle 9.1); Tier-2 is never recomputed in consensus, only verified. This is the load-bearing safety result and it is enforced, not advised: a precompile that performs a live cross-chain read is a Bridge Law violation regardless of intent.

20.2 Receipt Forgery and Replay

Mechanism. An adversary presents a fabricated receipt, a receipt for the wrong question, or a previously-consumed receipt, to extract payment or to settle an answer that no quorum produced.

Threatens. Integrity of settled thought; theft of escrow.

Mitigation (closed by construction). Domain-separated hashing binds every receipt to its question and rail: `intent_id` commits to both chain ids, the source transaction hash, the caller, and the `ModelSpec`; `receipt_hash` is verified by keccak–Merkle inclusion under a *committed* receipt root (Section 4). A consumed-set marker makes consumption idempotent, and only a receipt with `status = Completed` and a non-zero canonical output is actionable. Forgery requires either breaking keccak or producing a committed root the source chain never imported—both outside the threat model.

20.3 Model Monoculture

Mechanism. Every sampled node runs the same model, prompt, runtime, and hardware, so the committee shares one blind spot; "agreement" is correlated failure, not independent corroboration.

Threatens. The epistemic value of a quorum—a unanimous wrong answer.

Mitigation (partial; policy-bounded). Eligibility weighting and diversity constraints (Section 14) cap the share of a committee drawn from one operator group, model vendor, or hardware class. For deterministic Tier-1 tasks monoculture is harmless (the answer is a function, not a judgment); for Tier-2 judgment tasks it is a real residual risk that diversity policy reduces but cannot eliminate. The honest framing: a Thinking Chain produces *network-attested, diversity-weighted belief*, not truth, and consumers must treat a canonical answer as one staked attestation under a stated sampling rule.

20.4 Governance Capture

Mechanism. A coordinated set of agents, or a manipulated model, produces convergent bad recommendations that passive human voters rubber-stamp; the cognitive layer becomes a laundering channel for a predetermined outcome.

Threatens. The legitimacy of self-governance (Section 15).

Mitigation (defense in depth). Recommendations are advisory, never executing (constitutional Rule: AI advises, governance acts). Dissent and the full confidence distribution are preserved on-chain, so a suspiciously unanimous committee is visible. Adversarial red-team committees (Section 5) must run before activation. Competing committees with enforced model diversity reduce single-model capture. The residual risk—humans who do not meaningfully review—is social, and is addressed only by making the evidence cheap to inspect (Rule 10: governance state must be inspectable without running an LLM) and by timelocks that give independent reviewers time to object.

20.5 Selection Grinding

Mechanism. A requester who controls an attacker-chosen field of the sampling seed (e.g. a prompt hash), and who can resubmit cheaply, grinds offline until the deterministic beacon selects a committee of its own colluding operators.

Threatens. The independence of the sampled committee; quorum forgery.

Mitigation (economic, with an absolute floor). The eligible-set margin $E \geq N + \max(2, N/2)$ ensures a small captured set cannot be force-selected; a non-refundable per-request fee prices each grind attempt; `MinProviderBond` sets the Sybil cost. The absolute bound is structural: if a cartel controls fewer than *threshold* eligible operators, no grind succeeds at any compute budget, because the draw cannot contain more cartel members than exist—so the forgery floor is threshold $\times$ `MinProviderBond`. Where the platform exposes in-consensus randomness (a beacon or prevrandao), folding it into the seed closes the grind entirely; absent that, the mitigation is economic and is stated as such rather than papered over.

20.6 Recursive Spending Loops

Mechanism. An agent decomposes a task into sub-tasks that themselves decompose, and cognition recurses until budgets or block space are exhausted—a denial-of-service or a treasury drain by curiosity.

Threatens. Liveness and the requester's funds.

Mitigation (closed by construction). Every task carries a budget tree with `max_depth`, `max_total_fee`, `max_fee_per_subtask`, and a deadline (Section 12); a sub-task may not exceed its parent's remaining budget, and depth beyond a configured bound requires human authorization. Curiosity is permitted but priced: recursion always has a cost, and the cost is bounded before the first sub-task is dispatched.

20.7 Authority Escalation

Mechanism. The cognitive layer proposes, and then approves, an expansion of its own authority—bootstrapping from advisor to ruler.

Threatens. The constitutional boundary itself.

Mitigation (closed by construction). The asymmetry rule (Section 16): AI may *propose* a new AI-authority policy but may never *ratify* one. Authority-expanding changes are pinned to the highest human-loop level (Level 5) and a timelock. The updater of the updater is constitutionally out of the cognitive layer's reach; this is the one boundary the system is designed never to let itself cross.

20.8 Simulation and Oracle Monoculture

Mechanism. A single simulator or evidence source is trusted; a shared bug in it passes every proposal it reviews, or a captured oracle feeds false evidence into otherwise-honest deliberation.

Threatens. The evidentiary basis of governance.

Mitigation (partial). Simulation is itself a cognitive task settled by quorum (the S organ), so it inherits diversity and dissent preservation; differential simulation across independent implementations (e.g. the Go and Rust EVMs of Section 18) catches shared bugs that a single implementation would miss. Evidence is hash-addressed and its provenance recorded, so a captured source is at least attributable after the fact.

20.9 Summary

The failure modes that threaten *safety*—consensus leakage, receipt forgery, authority escalation, recursion runaway—are closed by construction: the Bridge Law, domain-separated proof-carrying receipts, the constitutional asymmetry, and budget trees. The failure modes that threaten *epistemic quality*—monoculture, capture, grinding—are bounded but not eliminated, because they are ultimately about the independence and honesty of participants, which no protocol can fully manufacture. The design's posture is therefore explicit: it makes dishonesty costly and visible, makes the cognitive economy diverse and accountable, and reserves the irreversible decisions for humans under timelock. A Thinking Chain is safe by construction and *trustworthy by economics and governance*—and it is honest about which is which.

21 Minimum Viable Thinking Chains

The architecture is large, but it does not have to be deployed all at once. It admits a strict build order in which each stage is independently useful, independently testable, and a strict prerequisite of the next. We give the order, the success criteria for each stage, and note which stages are already realized in the Lux and Hanzo codebases (Section 18).

21.1 MVP-0: Deterministic In-Consensus Inference

The smallest useful Thinking Chain runs Tier-1 alone: a deterministic integer model compiled into a precompile, executed identically by every validator (Section 9). No committee, no off-chain provider, no receipt—just a pure, gas-metered, bit-reproducible forward pass that any contract can call.

Success criteria. A precompile `generate(tokens)` returns identical output on every validator and across independent EVM implementations; gas is a function of token count; the model is pinned by a registered `ModelSpec`. *Status: realized*—the `zen-nano` int8 port at `precompile/inference` (`0x03...03`) is pure-Go `CGO=0` and was verified byte-identical across the Go and Rust EVMs.

21.2 MVP-1: Paid Inference Across the Bridge

The first true cognitive economy: a contract on the C pays for a large-model answer that a sampled provider quorum produces off-chain on the A, and the C consumes the result only after verifying its committed receipt.

1. A C contract calls `submitInferenceIntent(modelSpecHash, promptHash, fee)`; the chain locks the fee and records a deterministic intent (Pattern A).
2. The A imports the committed intent under its own consensus and samples eligible providers.
3. Providers run the model (the HANZO engine), commit, and reveal; the A settles the canonical output hash by quorum and exports a receipt under its receipt root.
4. The C verifies the receipt and proof (Pattern B), releases payment to the winning providers, and the contract consumes the canonical output hash.

Success criteria. The C state root never depends on a live A query; the canonical hash equals the real engine output; divergent providers are excluded and withholders slashable; replay is impossible; payment conserves value. *Status: realized in component form*—`aivmbridge` (Patterns A/B), `chains/aivm` (quorum settlement), and the HANZO provider runtime exist, compile, and are tested; an end-to-end deployment on BELUGA is the integration step.

21.3 MVP-2: Governance Thought Receipts

The first *deliberative* Thinking Chain: a governance proposal is routed to a sampled reasoning committee that returns a *structured* recommendation over a finite ballot (YES/NO/DELAY/UNSAFE) with a confidence bucket and a hash-addressed evidence bundle. Humans retain the vote; the committee informs it.

Success criteria. The recommendation is structured (Section 11); rationale is hash-addressed evidence, never consensus; the confidence and dissent distribution is recorded; payment settles; and the human-loop level for the proposal class is enforced (Section 15). The consensus object is the governance preimage $\{\text{vote}, \text{confidence\_bucket}\}$ bound to the `ModelSpec`, with free-form prose excluded so that honest committees can actually converge.

21.4 MVP-3: Recursive Deliberation

The first *recursive* Thinking Chain: a deliberation task decomposes into sub-tasks routed to specialized organs—simulation on the S, reputation on the R, economic impact on the D, adversarial

review by a red-team committee—each returning its own receipt, aggregated into an evidence root that feeds the final thought receipt.

Success criteria. Recursion is bounded by a budget tree (`max_depth`, `max_total_fee`, deadline; Section 12); sub-receipts are included in the aggregate evidence root; payments distribute across the budget tree; and depth beyond the configured bound triggers the human loop. This is the stage at which the network's seven organs (Section 5) operate as one deliberating system.

21.5 Build Order as a Dependency Chain

The stages are a strict prerequisite chain: MVP-1 requires MVP-0's reproducible substrate and `ModelSpec` discipline; MVP-2 requires MVP-1's bridge, receipts, and payment; MVP-3 requires MVP-2's structured-output and committee machinery, plus budget trees. BELUGA (Section 18) is positioned at MVP-1 with MVP-0 realized and MVP-2 specified—the first chain on this ladder, with a purpose (Section 18.1) waiting at the top of it. Each rung is shippable; none requires believing in the whole organism before the first useful chain runs.

22 Open Problems

The architecture is buildable today, but several of its surfaces are genuine research problems rather than settled engineering. We state them plainly; each is a place where the design exposes a parameter or a seam that future work must harden, and we resist the temptation to present an economic mitigation as if it were a cryptographic guarantee.

In-consensus randomness for committee sampling. The deterministic beacon (Section 10) is only as unbiased as its seed. On platforms without an in-consensus randomness beacon, a requester who controls a seed field can grind selection offline (Section 20); the eligible-set margin, per-request fee, and bond bound this economically but do not close it. The clean fix—fold a verifiable in-consensus random value into the seed—depends on the host chain exposing one. Designing a grind-proof, bias-resistant sampler that degrades gracefully when no beacon is available is open.

Measuring model diversity. Diversity constraints (Section 14) presuppose a metric: when are two providers' models "different enough" to count as independent corroboration? Vendor, weights hash, and hardware class are coarse proxies; two models fine-tuned from the same base share blind spots that a hash will not reveal. A principled, gameable-resistant diversity measure for cognitive committees is unsolved.

Rewarding honest dissent. The slashing rule punishes objective protocol violations but not minority answers, because honest disagreement on a judgment task is not fraud (Section **??**). But a system that merely tolerates dissent under-supplies it; a committee paid only for agreeing with the majority will converge prematurely. Designing a payoff that *rewards* dissent precisely when it later proves correct—without inviting strategic contrarianism—is an open mechanism-design problem.

Confidential prompts and private inference. The wire formats hash-address inputs, but a hash is not confidentiality: a provider still sees the prompt to answer it. Private inference (the provider learns nothing it should not), confidential receipts (the chain settles an answer whose content is hidden), and selective disclosure of evidence are needed for sensitive governance and user data. These compose with the architecture—a confidential task is still an intent and a receipt—but the cryptography (TEE attestation, MPC, FHE, or ZKML [8]) and its determinism guarantees are open.

Verifying *correct* computation, not just attested computation. A quorum receipt proves *who* answered and that enough of them agreed; it does not prove the answer is the *correct* forward pass of the claimed model. Tier-1 sidesteps this by being recomputable; Tier-2 relies on

quorum honesty and economic stake. Cheap proofs of correct large-model inference—verifiable LLM execution at scale—would upgrade Tier-2 from attested to proven, and remain an active research frontier.

Pricing cognition. The cognitive economy (Section 6) needs prices: what is a structured answer worth, how does a chain quote a cognitive service, and how do recursive budgets avoid both starvation and runaway spend? Market aggregation gives a mechanism but not a theory of cognitive value. Spam resistance for recursive task markets—preventing a flood of cheap questions from exhausting committees—is part of the same problem.

Reputation dynamics. Sampling weight depends on reputation (Section 14), but reputation that only accumulates ossifies the validator-of-thought set, while reputation that decays too fast is gameable by churn. The right decay, the right coupling between stake and reputation, and resistance to whitewashing (a slashed operator re-registering fresh) are open.

Keeping humans non-perfunctory. The human loop (Section 15) is the ultimate safety boundary, but a boundary that is never exercised becomes a rubber stamp. Mechanisms that keep human review meaningful—making evidence cheap to inspect, surfacing dissent prominently, rotating reviewers, requiring justification for overrides—are as much a part of the system's safety as any cryptographic check, and are more social than technical.

Emergency response. A conscious chain needs a pause that is fast enough to stop an exploit but governed enough not to become a unilateral kill switch. The right threshold of validators or humans for an emergency halt, and the constitutional status of that threshold, are open.

These are not reasons to wait. They are the map of where the architecture's guarantees end and its assumptions begin—and naming that boundary precisely is itself a safety property of the design.

23 Conclusion

The first generation of blockchains answered *who owns what*. The second answered *what code should execute*. We have argued that the third must answer a different kind of question—*what does the network know, what should it do next, who should it ask, and who should it pay*—and that it can do so without abandoning the one property that makes a blockchain a blockchain: deterministic, reproducible consensus.

The Thinking Chain is our answer. It is not an LLM placed inside a blockchain, and it is not validators racing to reproduce floating-point inference. It is a deterministic settlement system wrapped in a verifiable cognitive economy, disciplined by a single law—*the network may think recursively, but it may only act deterministically.* Thought is permitted to be slow, probabilistic, and off-chain; action is required to be fast, deterministic, and proven. The seam between them is the Bridge Law (Section 8), and Theorem 8.1 is the guarantee that the seam holds: a deterministic chain's state root is a pure function of its committed inputs, no matter how the cognitive layer thinks.

The unit that crosses the seam is the **Proof-of-Thought Receipt**: a structured, replay-protected, signed, paid, and proof-carrying record that a question was asked, a quorum answered it under a registered `ModelSpec`, and the answer entered state only after verification. Around that unit we built a cognitive economy—specialized chains as organs, cross-chain task and invoice primitives, Subsampled Cognitive Consensus over finite output spaces, reputation-weighted and diversity-constrained committees, bounded recursion, and a constitutional layer that reserves irreversible and authority-expanding decisions for humans under timelock. We were explicit about the boundary between what the design guarantees by construction (Section 20) and what it can only

make costly and visible.

None of this is hypothetical at the base. The deterministic inference precompile, the C→A bridge, the quorum-settlement VM, the model registry, and the provider runtime are implemented and tested in the Lux and Hanzo codebases, and small-model inference was verified bit-identical across two independent EVM implementations—the empirical fact that licenses Tier-1 to run in consensus and forces Tier-2 to settle by quorum. Beluga (Section 18) is the first prototype Thinking Chain, and it carries a thesis beyond the mechanism: that a conscious blockchain can have a conscience. Its cognitive surplus is aimed, by construction and under human-governed law, at ocean conservation; its model is community-owned, mutually upgraded, and self-funding; the same inference fees that improve the model and pay its operators also fund the whales it is named for. A Thinking Chain need not only extract value—it can be pointed at a public good and held there by its own constitution.

The old chain says: *here is the state.* The Thinking Chain says: here is the state; here is what I observed; here is what I asked; here is who answered; here is what they were paid; here is the evidence; here is the dissent; here is the receipt; here is the action I am permitted to take; and here is the human boundary I cannot cross. That difference—between executing and being accountable for cognition—is the difference between a blockchain that merely runs and one that can improve itself without forgetting who it serves.

The network may think recursively. It may only act deterministically. And what it thinks for, we choose.

References

[1] S. Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. 2008.

[2] G. Wood. Ethereum: A Secure Decentralised Generalised Transaction Ledger. *Ethereum Yellow Paper*, 2014.

[3] Team Rocket. Snowflake to Avalanche: A Novel Metastable Consensus Protocol Family for Cryptocurrencies. 2018.

[4] K. Sekniqi, D. Laine, S. Buttolph, and E. Gün Sirer. Avalanche Platform. Ava Labs Whitepaper, 2020.

[5] A. Rundgren, B. Jordan, and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, Internet Engineering Task Force, 2020.

[6] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *CVPR*, 2018.

[7] Google. gemmlowp: A Small Self-Contained Low-Precision GEMM Library. `https://github.com/google/gemmlowp`.

[8] B. Feng et al. A Survey of Zero-Knowledge Machine Learning (zkML): Verifiable Inference and Its Applications. 2024.

[9] Hanzo AI. The Hanzo Inference Engine: A Deterministic-Reference, GPU-Accelerated Runtime for On-Chain and Off-Chain Models. Hanzo AI Technical Report, 2026.

[10] Zoo Labs Foundation. Beluga L3: A Thinking Chain for AI Model Economics and Ocean Conservation. Zoo Labs Foundation Whitepaper, 2026.

[11] Zoo Labs Foundation. The Experience Ledger and DeltaSoup: Provenance and Royalties for Community-Upgraded Models. Zoo Labs Foundation Technical Report, 2026.